



Basi di Dati
Progetto A.A. 2025/2026

Bacheca elettronica di annunci

0309016

Matteo Marcoccia

Indice

1. Descrizione del Minimondo.....	2
2. Analisi dei Requisiti.....	3
3. Progettazione concettuale.....	8
4. Progettazione logica.....	21
5. Progettazione fisica.....	33
Indici.....	38

1. Descrizione del Minimondo

1 Bacheca elettronica di annunci

2
3 Si vuole realizzare un sistema informativo per la gestione di una bacheca elettronica di
4 annunci. Tale bacheca permette agli utenti del sistema di inserire annunci per la vendita di
5 materiale usato, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla
6 vendita/consegna dell'oggetto, o di inserire domande (in maniera pubblica) sull'oggetto.

7
8 Un utente del sistema si registra scegliendo un username univoco, inserendo tutte le sue
9 informazioni anagrafiche, indicando un indirizzo di residenza ed eventualmente un indirizzo
10 di fatturazione, un numero arbitrario di recapiti (telefono, cellulare, email) indicandone uno
11 come mezzo di comunicazione preferito.

12
13 I gestori del servizio possono creare una gerarchia di categorie per gli annunci. Un utente,
14 per creare un annuncio, seleziona una categoria e scrive una descrizione dell'oggetto.
15 Quando un oggetto inserito in bacheca è stato venduto, l'utente lo indica come tale e questo
16 non viene più visualizzato nella bacheca pubblica.

17
18 Un utente del sistema, una volta letto e scelto un annuncio, può decidere di inserire un
19 commento pubblico o di inviare un messaggio privato all'utente che ha inserito l'annuncio.
20 Similmente, un utente può "seguire" uno degli annunci, venendo così informato ogni volta
21 che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota).

22
23 I gestori del servizio possono generare un report indicante in forma aggregata per ciascun
24 utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come
25 venduti.

2. Analisi dei Requisiti

Identificazione dei termini ambigui e correzioni possibili

Linea	Termine	Nuovo termine	Motivo correzione
3	Sistema informativo	Sistema informatico	Con “sistema informatico” si vuole indicare in modo più specifico la componente software che gestisce e memorizza i dati della bacheca, distinguendola dal concetto più ampio di sistema informativo.
5, 6, 14, 15	Materiale usato/oggetto	Articolo	Il testo usa sia “materiale usato” che “oggetto” per riferirsi alla stessa cosa: si sceglie “articolo” come termine unico, più generico e riutilizzabile in tutta la specifica.
6	Domande	Commenti	“Domande” viene sostituito con “commenti” per uniformare la terminologia, in quanto il testo utilizza già l’espressione “commento pubblico” nella linea 19.
9	Informazioni anagrafiche	Dati Anagrafici (Nome, Cognome, Codice Fiscale, Data Nascita)	L’espressione “informazioni anagrafiche” è eccessivamente generica: vengono quindi indicati esplicitamente i dati richiesti in fase di registrazione.
10	Numero arbitrario di recapiti	Uno o più recapiti di tipologia distinta	L’espressione “numero arbitrario di recapiti” potrebbe indicare la possibilità di inserire più recapiti della stessa tipologia. Viene quindi sostituita con “uno o più recapiti di tipologia distinta” per specificare che l’utente può fornire al massimo un telefono fisso, un cellulare e un’e-mail, valorizzandone almeno uno.
10	Telefono	Telefono Fisso	Il termine “Telefono” viene precisato come “telefono fisso” per distinguerlo nettamente da “cellulare” ed evitare sovrapposizioni semantiche nella definizione dei tipi di recapito.
15,16	Bacheca/ Bacheca pubblica	Elenco degli annunci attivi	“Bacheca” e “bacheca pubblica” vengono usati nel testo con significati diversi: si adotta “elenco degli annunci attivi” per indicare in modo univoco la vista pubblica degli annunci disponibili.

Specificata disambiguata

Bacheca elettronica di annunci

Si vuole realizzare un **sistema informatico** per la gestione di una bacheca elettronica di annunci. Tale bacheca permette agli utenti del sistema di inserire annunci per la vendita di **articoli**, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla vendita/consegna dell'**articolo**, o di inserire **commenti** (in maniera pubblica) sull'**articolo**.

Un utente del sistema si registra scegliendo un username univoco, inserendo tutti i suoi **dati anagrafici** (**Nome, Cognome, Codice Fiscale, Data Nascita**), indicando un indirizzo di residenza ed eventualmente un indirizzo di fatturazione, **uno o più recapiti di tipologia distinta** (**telefono fisso, cellulare, email**) indicandone uno come mezzo di comunicazione preferito.

I gestori del servizio possono creare una gerarchia di categorie per gli annunci. Un utente, per creare un annuncio, seleziona una categoria e scrive una descrizione dell'**articolo**. Quando un **articolo** inserito nell'**elenco degli annunci attivi** è stato venduto, l'utente lo indica come tale e questo non viene più visualizzato nell'**elenco degli annunci attivi**.

Un utente del sistema, una volta letto e scelto un annuncio, può decidere di inserire un commento pubblico o di inviare un messaggio privato all'utente che ha inserito l'annuncio. Similmente, un utente può "seguire" uno degli annunci, venendo così informato ogni volta che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota).

I gestori del servizio possono generare un report indicante in forma aggregata per ciascun utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come venduti.

Glossario dei Termini

Termine	Descrizione	Sinonimi	Collegamenti
Utente	Persona fisica che si registra al sistema mediante username univoco per pubblicare annunci, interagire o seguire altri annunci	Utente del sistema	Dati anagrafici, Username, Indirizzo, Recapito, Annuncio, Messaggio privato, Commento pubblico, Seguire
Username	Codice alfanumerico univoco scelto	Nome utente, Identificativo utente	Utente

	dall'utente in fase di registrazione per identificarsi in modo univoco all'interno della piattaforma		
Gestore del servizio	Utente con privilegi gestionali autorizzato a creare la struttura delle categorie e a generare report sull'attività della piattaforma	Amministratore, admin	Categoria, Report, Utente
Annuncio	Inserzione pubblicata da un utente per la vendita di un articolo usufruendo delle categorie disponibili	Inserzione	Utente, Articolo, Categoria, Commento pubblico, Messaggio privato, Seguire, Nuova Nota
Articolo	Prodotto usato messo in vendita dall'utente attraverso la pubblicazione di un annuncio	Oggetto, materiale	Annuncio
Categoria	Classificazione tematica per organizzare e catalogare gli annunci pubblicati	Etichetta	Annuncio, Gestore del servizio, Gerarchia di categorie
Gerarchia di categorie	Relazione che collega una categoria a una categoria più generale (categoria padre), permettendo di organizzare le categorie in una struttura ad albero su più livelli	Struttura ad albero delle categorie	Categoria
Recapito	Informazione di contatto (telefono fisso, cellulare, email) fornita dall'utente in fase di registrazione, di cui uno deve essere contrassegnato come preferito	Mezzo di comunicazione, Contatto	Utente
Commento pubblico	Intervento o domanda visibile a tutti gli utenti del sistema, inserito in corrispondenza di uno specifico annuncio	Commento	Utente, Annuncio
Messaggio privato	Comunicazione riservata inviata da un	Messaggio, chat	Utente, Annuncio, Articolo

	utente a un altro per concordare i dettagli relativi alla vendita e alla consegna di un articolo.		
Nuova nota	Aggiornamento, modifica o integrazione inserita sull'annuncio che attiva le notifiche per gli utenti che lo seguono	Modifica, Aggiornamento	Utente, Annuncio, Seguire
Seguire	Funzionalità di monitoraggio attiva su un annuncio che consente all'utente di venire informato ad ogni modifica sull'annuncio	Follow, Monitoraggio	Utente, Annuncio, Nuova nota
Elenco degli annunci attivi	Vista pubblica degli annunci disponibili la cui vendita non è ancora stata completata	Bacheca pubblica	Annuncio
Report	Calcolo aggregato generato dai gestori indicante, per ciascun utente, la percentuale di annunci venduti rispetto al totale degli annunci pubblicati	Report aggregato, Report % vendite	Gestore del servizio, Utente, Annuncio

Raggruppamento dei requisiti in insiemi omogenei

Frasi di carattere generale

Si vuole realizzare un sistema informatico per la gestione di una bacheca elettronica di annunci. Tale bacheca permette agli utenti del sistema di inserire annunci per la vendita di articoli, di scambiare messaggi tra di loro (in maniera privata) per accordarsi sulla vendita/consegna dell'articolo, o di inserire commenti (in maniera pubblica) sull'articolo.

Frasi relative agli utenti

Un utente del sistema si registra scegliendo un username univoco, inserendo tutti i suoi dati anagrafici (Nome, Cognome, Codice Fiscale, Data Nascita), indicando un indirizzo di residenza ed

eventualmente un indirizzo di fatturazione, uno o più recapiti di tipologia distinta (telefono fisso, cellulare, email) indicandone uno come mezzo di comunicazione preferito.

Frase relative ai gestori del servizio

I gestori del servizio possono creare una gerarchia di categorie per gli annunci. I gestori del servizio possono generare un report indicante in forma aggregata per ciascun utente del sistema quale percentuale di annunci pubblicati sono stati contrassegnati come venduti.

Frase relative agli annunci

Un utente, per creare un annuncio, seleziona una categoria e scrive una descrizione dell'articolo. Quando un articolo inserito nell'elenco degli annunci attivi è stato venduto, l'utente lo indica come tale e questo non viene più visualizzato nell'elenco degli annunci attivi.

Frase relative alla comunicazione tra utenti

Un utente del sistema, una volta letto e scelto un annuncio, può decidere di inserire un commento pubblico o di inviare un messaggio privato all'utente che ha inserito l'annuncio.

Frase relative al seguire e alle notifiche

Similmente, un utente può "seguire" uno degli annunci, venendo così informato ogni volta che su questo viene effettuata una modifica (ad esempio, viene inserita una nuova nota).

3. Progettazione concettuale

Costruzione dello schema E-R

Per la costruzione dello schema E-R finale è stata adottata una strategia Inside-Out. Tale approccio prevede di partire dall'individuazione del concetto centrale del sistema (il "nucleo" del minimondo) per poi procedere con espansioni successive verso la periferia. Ogni passo della modellazione prende le mosse dai raggruppamenti di requisiti omogenei definiti nella fase di analisi e viene accompagnato dal relativo schema E-R parziale, fino al raggiungimento dello schema concettuale completo.

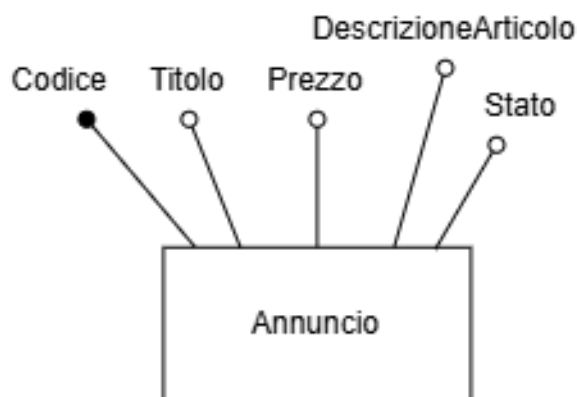
Passo 1: Individuazione del Nucleo

In linea con la strategia Inside-Out, il primo passo consiste nell'isolare l'oggetto attorno a cui ruota l'intero funzionamento della bacheca elettronica: l'entità "Annuncio". L'entità "Annuncio" possiede i seguenti attributi:

- Codice: attributo identificatore primario (chiave univoca di "Annuncio").
- Titolo: attributo stringa che definisce l'intestazione sintetica dell'annuncio.
- Prezzo: attributo numerico espresso in valuta che indica il costo dell'articolo in vendita.
- DescrizioneArticolo: attributo testuale che raccoglie la descrizione del bene messo in vendita.
- Stato: attributo che traccia la disponibilità dell'annuncio (valorizzabile come "InVendita" o "Venduto").

Scelte di progettazione:

- 1) Accorpamento dell'entità "Articolo": Sebbene il termine "Articolo" sia stato individuato nel glossario come concetto a se stante, si è scelto di non modellarlo come entità distinta in relazione 1:1 con l'annuncio. L'articolo risulta privo di un ciclo di vita indipendente e non possiede attributi propri oltre alla sua descrizione. Viene pertanto rappresentato direttamente all'interno di "Annuncio" tramite l'attributo testuale "DescrizioneArticolo".
- 2) Assunzioni sugli attributi "Titolo" e "Prezzo": Sebbene non menzionati esplicitamente nella traccia disambiguata, si assume l'introduzione di un titolo e di un prezzo in quanto elementi fondamentali per la caratterizzazione di un qualsiasi annuncio di vendita in un contesto reale.
- 3) Modellazione dell'Elenco degli annunci attivi: L'elenco degli annunci attivi (o bacheca pubblica) non costituisce un'entità a se stante per evitare ridondanze. Esso viene modellato come una vista logica/derivata ottenuta applicando all'entità "Annuncio" il filtro Stato="InVendita". Quando un articolo viene venduto, l'utente aggiorna lo Stato in "Venduto", rimuovendolo dalla vista pubblica senza dover eliminare il record dal database.



Passo 2: Espansione verso gli Utenti

Il secondo passo amplia lo schema concettuale introducendo l'attore principale che interagisce con la bacheca: l'Utente, unitamente alla gestione dei suoi recapiti e alla relazione di pubblicazione degli annunci.

L'entità "Utente" rappresenta ogni persona fisica registrata nel sistema, con i seguenti attributi:

- Username: Stringa univoca scelta in fase di registrazione (chiave univoca di "Utente")
- Dati anagrafici: Attributo composto dai seguenti attributi: (Nome, Cognome, CodiceFiscale, DataNascita).
- IndirizzoResidenza: Stringa
- IndirizzoFatturazione: Stringa opzionale
- Recapito: Attributo composto, contenente 3 attributi opzionali: TelefonoFisso, Cellulare, Email
- RecapitoPreferito: un attributo che può avere come valore (TelefonoFisso, Cellulare, Email)

Scelte di progettazione:

1. Struttura del Recapito e mezzo di comunicazione preferito: si è scelto di rappresentare i tre possibili canali di contatto come sotto-attributi distinti e singolarmente opzionali di un unico attributo composto "Recapito", anziché come attributo multivalore generico. Questa scelta garantisce implicitamente che un utente possa avere al più un valore per ciascuna tipologia di contatto, permettendo così all'attributo atomico "RecapitoPreferito" (valorizzabile come "TelefonoFisso", "Cellulare" o "Email") di identificare in modo univoco e non ambiguo anche il valore specifico da utilizzare.
2. Integrità tra recapiti e mezzo preferito (Regola aziendale): Il modello E-R non permette di esprimere nativamente un vincolo per cui il valore di RecapitoPreferito debba corrispondere a una tipologia di contatto effettivamente registrata dall'utente tra i sotto-attributi valorizzati di

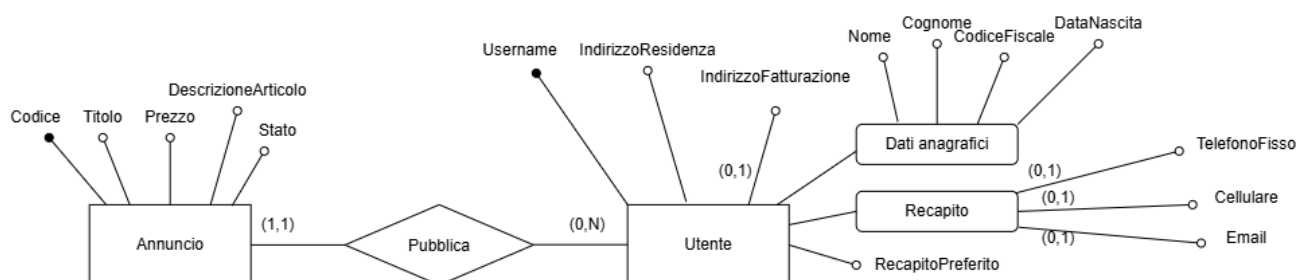
“Recapito”. Questo vincolo d’integrità viene formalizzato mediante le Regole Aziendali RV1 e RV2.

- Scelta relativa alle credenziali di accesso: la password dell’utente non viene rappresentata nello schema E-R, poiché la specifica richiede esclusivamente la registrazione mediante uno username univoco e non descrive un meccanismo di autenticazione. Per consentire il funzionamento concreto dell’applicazione, nella progettazione fisica viene tuttavia introdotto nella tabella Utente l’attributo tecnico PasswordHash, contenente esclusivamente una rappresentazione crittografica della password. La password originale non viene mai memorizzata nel database.

Passo 3: Associazione “Pubblica”

Collega il nucleo “Annuncio” con l’entità “Utente”:

- Cardinalità su “Utente”: (0,N)- Un utente registrato può aver pubblicato da 0 a N annunci.
- Cardinalità su “Annuncio”: (1,1)- Ogni annuncio deve essere stato pubblicato esattamente da un determinato utente autore.



Passo 4: Espansione verso le Categorie e la Gerarchia

In questo passo lo schema concettuale viene esteso introducendo la struttura di catalogazione ed organizzazione degli annunci gestita dagli utenti amministratori.

L’entità “Gestore” come specializzazione dell’entità Utente:

Entità Padre: Utente.

Entità Figlia: Gestore.

L’entità “Categoria” rappresenta la classificazione tematica o merceologica in cui gli utenti inseriscono i propri annunci, e possiede i seguenti attributi:

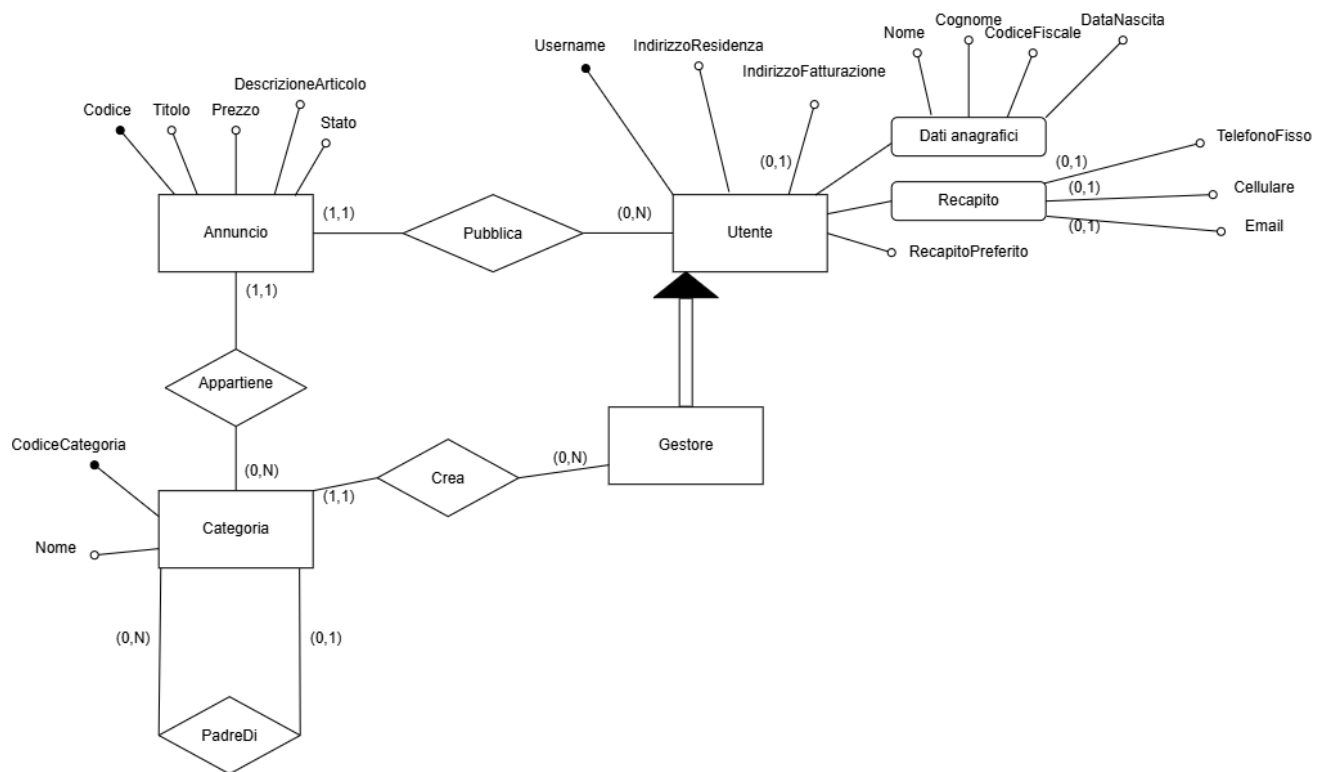
- CodiceCategoria: Attributo identificatore primario (chiave univoca di “Categoria”).
- Nome: Attributo stringa che definisce la denominazione della categoria.

Modellazione delle associazioni:

- Associazione ricorsiva “PadreDi” (Gerarchia di categorie): Collega l’entità “Categoria” con se stessa per modellare la sua struttura gerarchica ad albero.
 - Cardinalità su Categoria Padre: (0,N)- Una categoria può fungere da padre per 0 o più sottocategorie.
 - Cardinalità su Categoria Figlia: (0,1)- Una sottocategoria può essere contenuta in al più una sola categoria padre.
- Associazione “Appartiene”:
 - Cardinalità su “Annuncio”: (1,1)- Ogni annuncio deve essere associato obbligatoriamente a esattamente una categoria.
 - Cardinalità su “Categoria”: (0,N)- Una categoria può non contenere ancora alcun annuncio inserito o poterne contenere un numero arbitrario N.
- Associazione “Crea”: Tale associazione formalizza il vincolo per cui la struttura di catalogazione può essere creata e gestita unicamente dagli utenti con privilegi da gestore.
 - Cardinalità su “Gestore”: (0,N)- Un gestore può creare fino a N categorie.
 - Cardinalità su “Categoria”: (1,1)- Una categoria viene creata da un singolo gestore.

Scelte di progettazione:

1. Specializzazione dell’Utente e Gestore del Servizio: Si è introdotta una gerarchia parziale per modellare l’entità Gestore come specializzazione di Utente. Questa scelta consente di evitare la duplicazione dei dati anagrafici e di contatto condivisi da entrambi i soggetti, permettendo al contempo di associare formalmente al solo gestore le operazioni e i privilegi amministrativi previsti dalle specifiche (come la gestione delle categorie e la reportistica).
2. Gerarchia ricorsiva ad albero a profondità arbitraria: Anziché creare due entità distinte ("Categoria" e "Sottocategoria") che avrebbero rigidamente limitato la classificazione a soli due livelli, l’auto-associazione (1:N) consente ai gestori di creare alberi di categorie con qualsiasi livello di annidamento desiderato (es. Elettronica -> Informatica -> Computer -> Notebook).
3. Analisi dei Report e delle Funzioni Amministrative: Coerentemente con quanto definito nei requisiti, il calcolo aggregato relativo alla percentuale di vendite non viene rappresentato come un’entità indipendente nel modello concettuale. Trattandosi di un dato derivato ed elaborato partendo dalle informazioni residenti in Annuncio e Utente, la sua realizzazione avverrà a livello di progettazione logica e fisica. Nello specifico, si ricorrerà all’utilizzo di una Vista logica o di una Stored Procedure dedicata, la cui esecuzione sarà subordinata all’entità “Gestore”.



Passo 5: Espansione verso la comunicazione tra utenti

Questa fase della modellazione estende lo schema concettuale includendo le modalità di interazione tra i soggetti del sistema, operando una netta distinzione tra i flussi comunicativi di natura pubblica e i messaggi privati intercorsi tra venditore e potenziale acquirente.

L'entità **“Commento Pubblico”** rappresenta le osservazioni o le richieste di chiarimento accessibili a tutti gli utenti e inserite a corredo dell'annuncio, ed è caratterizzata dai seguenti attributi:

- CodiceCommento: attributo identificatore primario (chiave univoca dell'entità).
- Testo: attributo di tipo stringa che ospita il contenuto informativo della nota pubblica.
- DataOra: attributo temporale che registra l'istante preciso di pubblicazione.

L'entità **“Messaggio Privato”** descrive le interazioni confidenziali finalizzate alla definizione dei termini di transazione e logistica, definita dai seguenti attributi:

- CodiceMessaggio: attributo identificatore primario (chiave univoca dell'entità).
- Testo: attributo testuale che contiene il corpo della comunicazione riservata.
- DataOra: attributo temporale indicante il momento della trasmissione del messaggio.

Modellazione delle associazioni:

Associazione **“InserisceCommento”**:

- Cardinalità su **“Utente”**: (0,N)- Un utente registrato può scrivere da 0 a N commenti pubblici.
- Cardinalità su **“Commento Pubblico”**: (1,1)- Ogni commento pubblico è inserito esattamente da un determinato utente autore.

Associazione “RiferitoA”:

- Cardinalità su “Annuncio”: (0,N)- Un annuncio può ricevere da 0 a N commenti pubblici.
- Cardinalità su “Commento Pubblico”: (1,1)- Ogni commento pubblico è riferito obbligatoriamente a un singolo annuncio.

Associazione “Invia”:

- Cardinalità su “Utente”: (0,N)- Un utente registrato può inviare da 0 a N messaggi privati.
- Cardinalità su “Messaggio Privato”: (1,1)- Ogni messaggio privato deve essere inviato esattamente da un determinato utente mittente.

Associazione “Riceve”:

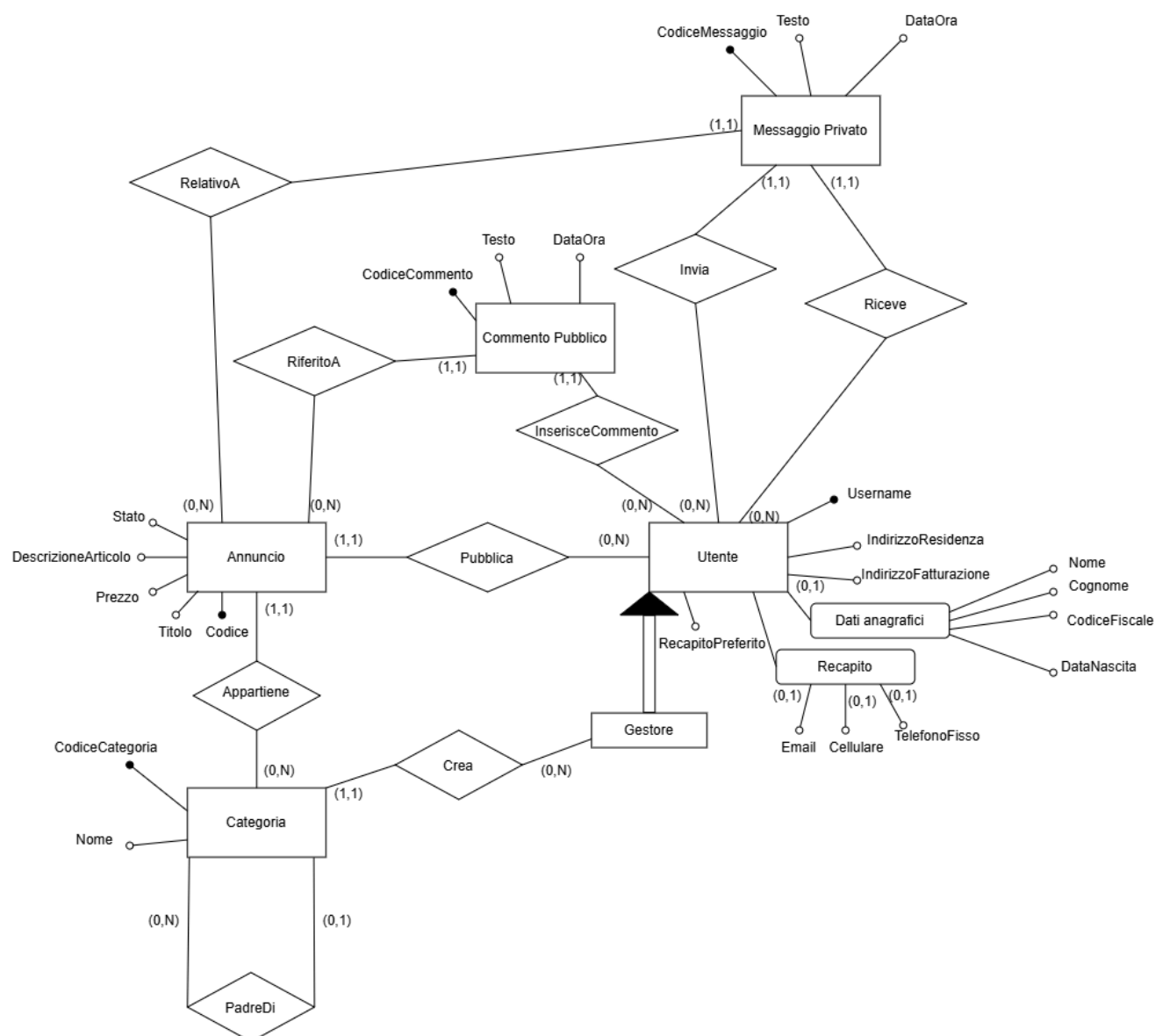
- Cardinalità su “Utente”: (0,N)- Un utente registrato può ricevere da 0 a N messaggi privati.
- Cardinalità su “Messaggio Privato”: (1,1)- Ogni messaggio privato è indirizzato obbligatoriamente a un singolo utente destinatario.

Associazione “RelativoA”:

- Cardinalità su “Annuncio”: (0,N)- Un annuncio può essere oggetto di 0 o N messaggi privati di trattativa.
- Cardinalità su “Messaggio Privato”: (1,1)- Ogni messaggio privato è riferito tassativamente a un determinato annuncio per cui si richiede l'accordo.

Scelte di progettazione:

1. Distinzione tra interazioni pubbliche e private: al fine di evitare ambiguità modellative, si è optato per la creazione di due entità indipendenti. Tale soluzione consente di gestire in modo atomico i commenti visibili alla collettività degli utenti, legati a un unico autore, e di isolare i messaggi di natura riservata, i quali esigono la specifica individuazione di mittente e destinatario per la tutela della confidenzialità nelle fasi negoziali.



Passo 6: Espansione verso le notifiche

In questa sesta fase della modellazione, lo schema concettuale viene completato mediante l'introduzione della funzionalità di monitoraggio degli annunci e del sistema di notifica attivato dall'inserimento di una nuova nota, dalla modifica delle informazioni principali di un annuncio o dal suo aggiornamento allo stato "Venduto".

L'entità "**Nuova Nota**" modella le integrazioni o le rettifiche apportate all'inserzione dal rispettivo venditore, definita dai parametri sottostanti:

- CodiceNota: attributo identificatore primario (chiave univoca dell'entità).
- TestoNota: attributo di tipo stringa contenente le specifiche dell'aggiornamento.
- DataOra: attributo temporale che registra il momento della pubblicazione.

Modellazione delle associazioni:

Associazione “Segui”:

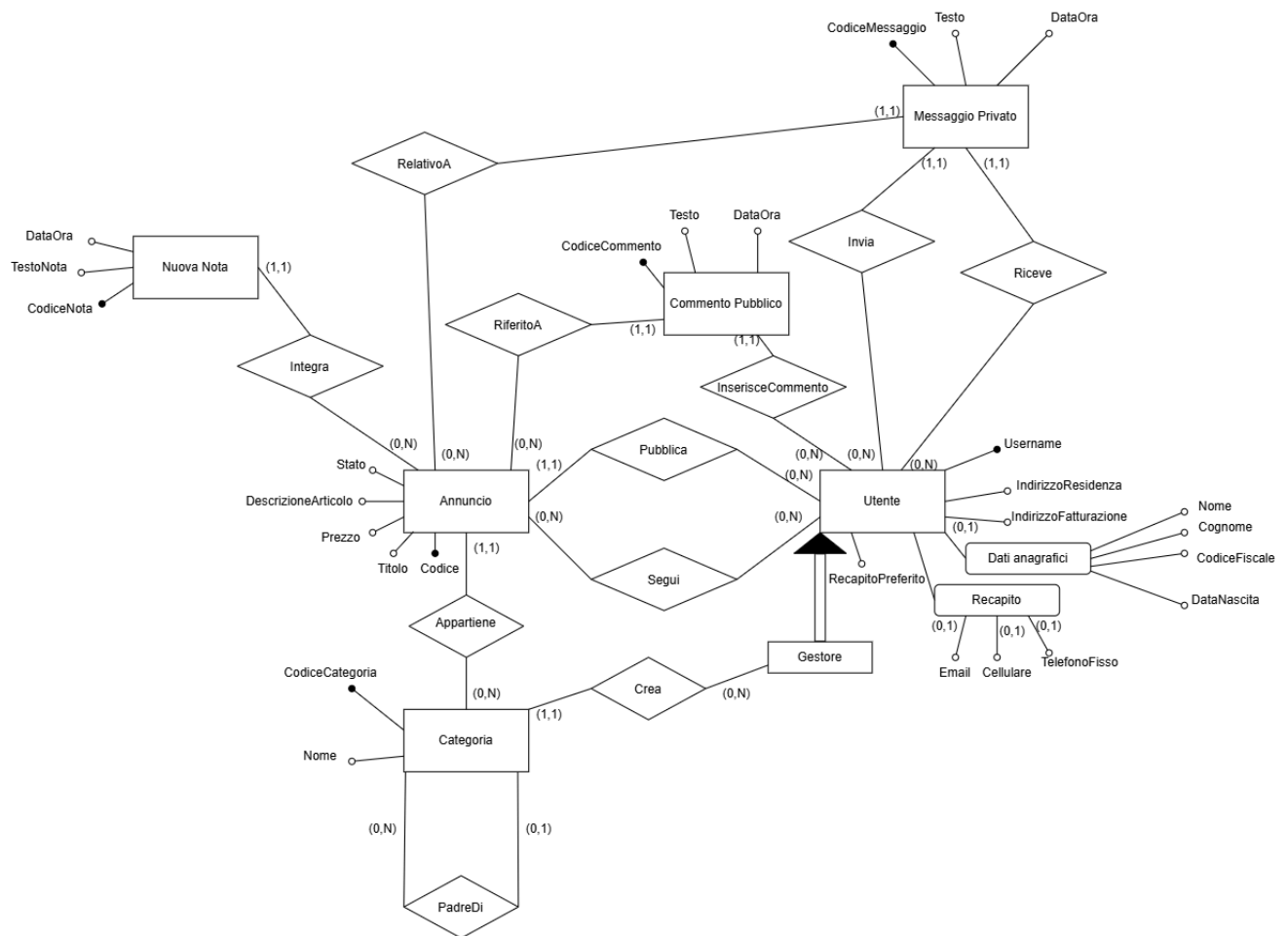
- Cardinalità su “Utente”: (0,N)- Un soggetto può monitorare un numero arbitrario di inserzioni.
- Cardinalità su “Annuncio”: (0,N)- Ogni inserzione può essere seguita da una moltitudine di utenti.

Associazione “Integra”:

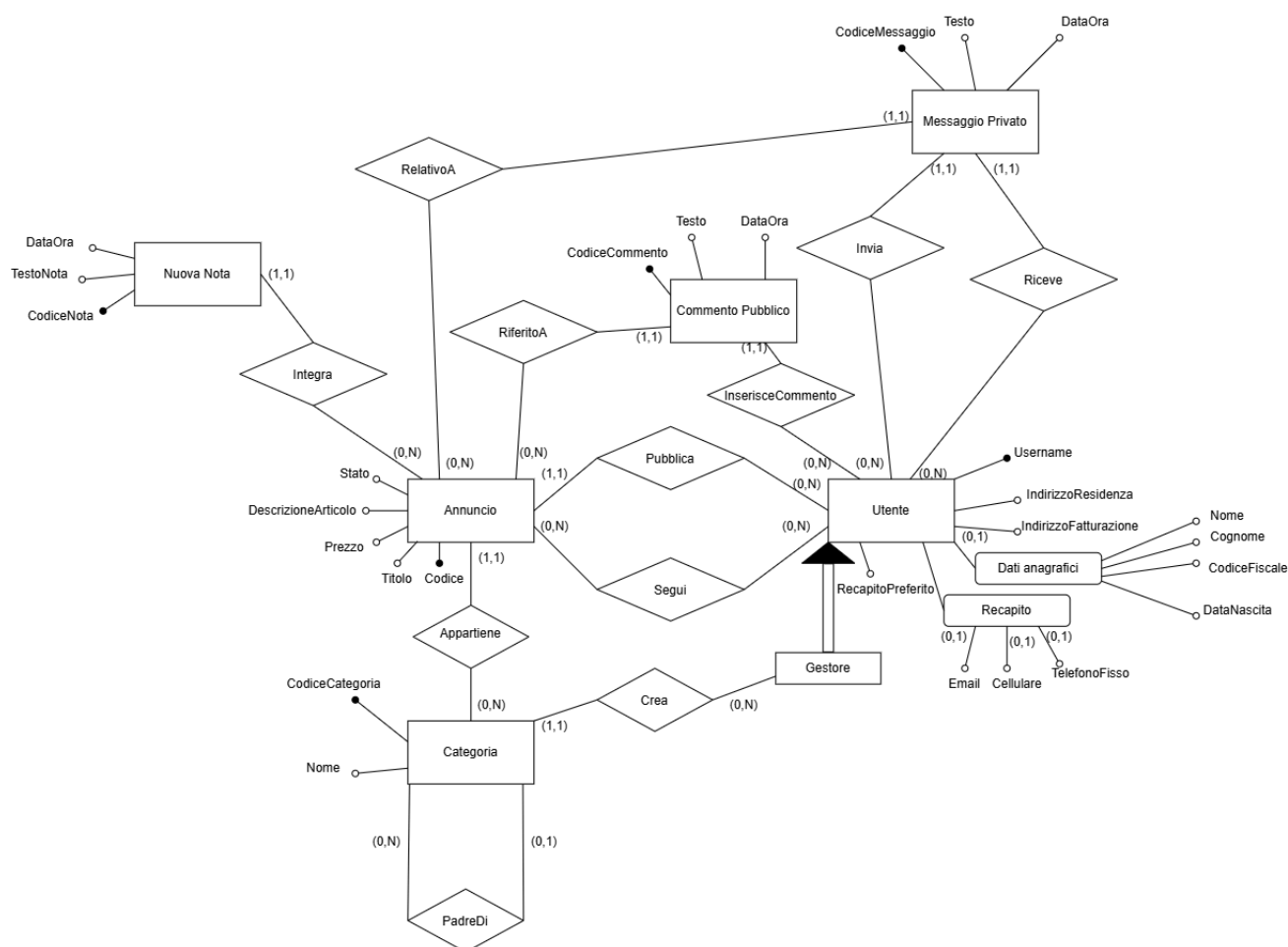
- Cardinalità su “Annuncio”: (0,N)- Un annuncio può accogliere nel tempo molteplici note integrative.
- Cardinalità su “Nuova Nota”: (1,1)- Ciascun aggiornamento è vincolato univocamente a un determinato annuncio.

Scelte di progettazione:

1. Modellazione delle Notifiche: la notifica non viene modellata come entità autonoma, poiché il sistema non deve conservarne lo storico né registrarne lo stato di lettura. Quando si verifica un evento notificabile, ossia l’inserimento di una nuova nota, la modifica delle informazioni principali di un annuncio oppure il suo aggiornamento allo stato “Venduto”, le stored procedure recuperano dalla relazione “Segui” gli utenti interessati, insieme al tipo e al valore del relativo recapito preferito. L’applicazione distingue quindi le notifiche in base al recapito selezionato (Email, Cellulare o TelefonoFisso) e ne simula l’invio mediante la scrittura di un messaggio nel log. L’invio effettivo tramite servizi esterni non viene implementato, poiché non rientra negli obiettivi del progetto.
2. Identificazione dell'autore per derivazione logica: non viene introdotta un’associazione diretta tra Nuova Nota e Utente, poiché ogni nota può essere inserita esclusivamente dall’autore dell’annuncio a cui è associata. L’autore della nota è quindi ricavato univocamente attraverso il percorso Nuova Nota → Annuncio → Utente. Il controllo dell’identità dell’utente che esegue l’operazione è demandato alla logica applicativa o alla stored procedure utilizzata per l’inserimento della nota.



Integrazione finale



Regole aziendali

Regole di Vincolo(RV)

- **RV1** – Un utente deve possedere almeno un recapito valorizzato tra Email, Cellulare e TelefonoFisso.
- **RV2** – Il valore di RecapitoPreferito deve appartenere all'insieme {Email, Cellulare, TelefonoFisso} e il recapito corrispondente deve essere valorizzato.
- **RV3** – Una categoria non può essere, direttamente o indirettamente tramite l'associazione ricorsiva PadreDi, padre di se stessa.
- **RV4** – Il mittente di un Messaggio Privato deve essere diverso dal destinatario del medesimo messaggio.
- **RV5** – L'utente che crea una Categoria deve appartenere alla specializzazione Gestore.

- **RV6** – In ogni Messaggio Privato relativo a un Annuncio, almeno uno tra mittente e destinatario deve coincidere con l'autore dell'annuncio.
- **RV7** – L'attributo Stato di un Annuncio può assumere esclusivamente i valori "InVendita" e "Venduto". Al momento della pubblicazione di un nuovo annuncio, il suo stato iniziale deve essere "InVendita".
- **RV8** – Al momento della registrazione, un nuovo Utente non può appartenere alla specializzazione Gestore.
- **RV9** – Non possono esistere due categorie con lo stesso nome e la stessa categoria padre. La regola si applica anche alle categorie radice, per le quali la categoria padre è nulla.
- **RV10** – L'autore di un annuncio non può seguire il proprio annuncio.
- **RV11** – Soltanto l'autore può aggiungere note integrative, modificare le informazioni o contrassegnare come venduto un proprio annuncio. Tali operazioni sono consentite esclusivamente quando l'annuncio si trova nello stato "InVendita".
- **RV12** – La generazione del report statistico complessivo è riservata agli utenti appartenenti alla specializzazione Gestore.
- **RV13** – Un utente può iniziare a seguire esclusivamente un annuncio nello stato "InVendita". La disiscrizione rimane consentita indipendentemente dallo stato dell'annuncio.
- **RV14** – Il CodiceFiscale deve essere composto da esattamente 16 caratteri alfanumerici maiuscoli.
- **RV15** – Se valorizzato, Email deve rispettare un formato sintatticamente valido, contenente una parte locale, il simbolo " @" e un dominio dotato di estensione.
- **RV16** – Se valorizzati, Cellulare e TelefonoFisso devono contenere da 7 a 15 cifre, eventualmente precedute dal simbolo "+".
- **RV17** – La DataNascita di un Utente non può essere successiva alla data corrente.
- **RV18** – Il Prezzo di un Annuncio deve essere strettamente maggiore di zero.
- **RV19** – Gli attributi testuali obbligatori non possono essere vuoti né composti esclusivamente da spazi. La regola si applica a Nome, Cognome e IndirizzoResidenza di Utente; Nome di Categoria; Titolo e DescrizioneArticolo di Annuncio; TestoNota di Nuova_Nota; Testo di Commento_Pubblico; Testo di Messaggio_Privato.
- **RV20** – Dopo che un annuncio è stato contrassegnato come "Venduto", non è più possibile inserire nuovi commenti pubblici né avviare nuove conversazioni private relative all'annuncio. Le conversazioni private già avviate possono proseguire e tutti i contenuti precedenti rimangono consultabili.

Regole di Derivazione (RD)

- **RD1** – La percentuale di annunci venduti da un determinato utente si calcola dividendo il numero dei suoi annunci aventi Stato = “Venduto” per il numero totale degli annunci da lui pubblicati e moltiplicando il risultato per 100. Se l’utente non ha pubblicato alcun annuncio, la percentuale viene convenzionalmente posta uguale a 0.

Dizionario dei dati

Entità	Descrizione	Attributi	Identificatori
Utente	Rappresenta un utente registrato abilitato a pubblicare annunci, inviare/ricevere messaggi privati, inserire commenti e seguire annunci.	Username, IndirizzoResidenza, IndirizzoFatturazione (opzionale), RecapitoPreferito, Dati anagrafici (composto: Nome, Cognome, CodiceFiscale, DataNascita), Recapito (composto: Email, Cellulare, TelefonoFisso)	Username
Gestore	Specializzazione di Utente che rappresenta gli amministratori autorizzati a creare la struttura delle categorie e a generare il report statistico complessivo.	Eredita tutti gli attributi dell'entità genitore Utente.	Username (ereditato)
Annuncio	Rappresenta l'inserzione di vendita di un articolo pubblicata da un utente all'interno della piattaforma.	Codice, Titolo, DescrizioneArticolo, Prezzo, Stato	Codice
Categoria	Rappresenta la classificazione tematica degli annunci, organizzabile gerarchicamente ad albero.	CodiceCategoria, Nome	CodiceCategoria
Commento Pubblico	Rappresenta un intervento testuale visibile pubblicamente inserito da un utente in calce a un annuncio.	CodiceCommento, Testo, DataOra	CodiceCommento

Messaggio Privato	Rappresenta una comunicazione riservata scambiata direttamente tra due utenti nell'ambito della trattativa per un annuncio.	CodiceMessaggio, Testo, DataOra	CodiceMessaggio
Nuova Nota	Rappresenta un'integrazione o aggiornamento testuale cronologico pubblicato dall'autore in calce al proprio annuncio.	CodiceNota, TestoNota, DataOra	CodiceNota

4. Progettazione logica

Volume dei dati

Concetto nello schema	Tipo ¹	Volume atteso
Utente	E	50.000
Gestore	E	10
Categoria	E	100
Annuncio	E	200.000
Commento Pubblico	E	600.000
Messaggio Privato	E	1.500.000
Nuova Nota	E	100.000
Pubblica	R	200.000
Appartiene	R	200.000
Crea	R	100
PadreDi	R	80
InserisceCommento	R	600.000
RiferitoA (Commenti)	R	600.000
Invia	R	1.500.000
Riceve	R	1.500.000
RelativoA (Messaggi)	R	1.500.000
Integra	R	100.000
Segui	R	400.000

Tavola delle operazioni

Cod.	Descrizione	Frequenza attesa
OP1	Registrazione di un nuovo utente e inserimento dei relativi recapiti	50/giorno
OP2	Pubblicazione di un nuovo annuncio associato a una categoria	1.000/giorno
OP3	Pubblicazione di una nuova nota integrativa per un annuncio con individuazione dei follower da notificare	500/giorno
OP4	Ricerca e navigazione annunci attivi per categoria e relative sottocategorie	40.000/giorno
OP5	Visualizzazione della scheda dettaglio di un annuncio (con note e commenti)	60.000/giorno
OP6	Inserimento di un commento pubblico sotto un annuncio	3.000/giorno
OP7	Invio di un messaggio privato di trattativa relativo a un annuncio	8.000/giorno
OP8	Iscrizione o disiscrizione dal monitoraggio di un annuncio (Segui)	2.500/giorno

¹ Indicare con E le entità, con R le relazioni

OP9	Modifica delle informazioni di un annuncio attivo (titolo, prezzo, descrizione) da parte dell'autore con individuazione dei follower da notificare	300/giorno
OP10	Aggiornamento dello stato di un annuncio a "Venduto" e rimozione dall'elenco degli annunci attivi con individuazione dei follower da notificare	400/giorno
OP11	Creazione e collocazione nell'albero gerarchico di una nuova categoria da parte di un gestore	2/mese
OP12	Generazione da parte del gestore del report statistico aggregato per utente con la percentuale di annunci venduti rispetto al totale pubblicato	1/settimana
OP13	Recupero dei dati necessari all'autenticazione e alla verifica del profilo	30.000/giorno
OP14	Visualizzazione degli annunci pubblicati da un utente ("i miei annunci")	5.000/giorno
OP15	Visualizzazione della conversazione privata tra due utenti relativa a un annuncio	15.000/giorno

Costo delle operazioni

Nel calcolo vengono considerati anche gli accessi logici effettuati implicitamente dal DBMS per verificare chiavi primarie, vincoli di unicità e integrità referenziale. Tali accessi possono quindi non corrispondere a istruzioni SELECT esplicitamente presenti nelle stored procedure.

OP1 – Registrazione di un nuovo utente e inserimento dei recapiti

- **Frequenza:** 50/giorno
- **Accessi logici:** 2 letture su Utente per verificare che Username e CodiceFiscale non siano già presenti; 1 scrittura su Utente per memorizzare i dati anagrafici, gli indirizzi e i recapiti.
- **Costo unitario:** $2R + 2(1W) = 4$ accessi
- **Costo complessivo:** $4 * 50 = 200$ accessi/giorno

OP2 – Pubblicazione di un nuovo annuncio

- **Frequenza:** 1.000/giorno
- **Accessi logici:** 1 lettura su Utente per verificare l'esistenza dell'autore; 1 lettura su Categoria per verificare l'esistenza della categoria selezionata; 1 scrittura su Annuncio, contenente anche i riferimenti all'autore e alla categoria.
- **Costo unitario:** $2R + 2(1W) = 4$ accessi

- **Costo complessivo:** $4 \times 1.000 = 4.000$ accessi/giorno

OP3 – Pubblicazione di una nuova nota integrativa con individuazione dei follower

- **Frequenza:** 500/giorno
- **Accessi logici:** 1 lettura su Annuncio per verificarne l'esistenza, controllare che lo Stato sia "InVendita" e accertare che l'utente richiedente coincida con l'Autore; 1 scrittura su Nuova_Nota; 4 letture per recuperare i follower e i rispettivi recapiti preferiti, stimando mediamente 2 follower per annuncio: 2 accessi a Segui e 2 accessi a Utente.
- **Costo unitario:** $5R + 2(1W) = 7$ accessi
- **Costo complessivo:** $7 \times 500 = 3.500$ accessi/giorno

OP4 – Ricerca e navigazione annunci attivi per categoria e relative sottocategorie

- **Frequenza:** 40.000/giorno
- **Accessi logici:** mediamente 2 letture su Categoria per individuare la categoria selezionata e attraversarne la gerarchia; mediamente 50 letture su Annuncio per recuperare gli annunci attivi appartenenti alla categoria selezionata e alle relative sottocategorie. Si assume che ogni ricerca restituisca mediamente 50 annunci aventi Stato = "InVendita".
- **Costo unitario:** $52R = 52$ accessi
- **Costo complessivo:** $52 \times 40.000 = 2.080.000$ accessi/giorno

OP5 – Visualizzazione della scheda dettaglio di un annuncio

- **Frequenza:** 60.000/giorno
- **Accessi logici:** 1 lettura su Annuncio per recuperare i dati dell'inserzione; 1 lettura su Categoria per recuperare il nome della categoria; 1 lettura su Utente per recuperare i dati pubblici e il recapito preferito dell'autore; mediamente 0,5 letture su Nuova_Nota, considerando 100.000 note distribuite su 200.000 annunci; mediamente 3 letture su Commento_Pubblico, considerando 600.000 commenti distribuiti su 200.000 annunci.
- **Costo unitario:** $6,5R = 6,5$ accessi
- **Costo complessivo:** $6,5 \times 60.000 = 390.000$ accessi/giorno

OP6 – Inserimento di un commento pubblico

- **Frequenza:** 3.000/giorno
- **Accessi logici:** 1 lettura su Utente per verificare l'esistenza dell'autore del commento; 1 lettura su Annuncio per verificarne l'esistenza e controllare che si trovi nello stato "InVendita"; 1 scrittura su Commento_Pubblico; 4 letture per recuperare mediamente 2 follower e i relativi recapiti preferiti, di cui 2 su Segui e 2 su Utente; 1 ulteriore lettura su Utente per recuperare il recapito preferito dell'autore dell'annuncio, se diverso dall'autore del commento.
- **Costo unitario:** $7R + 2(1W) = 9$ accessi

- **Costo complessivo:** $9 \times 3.000 = 27.000$ accessi / giorno

OP7 – Invio di un messaggio privato di trattativa

- **Frequenza:** 8.000/giorno
- **Accessi logici:** 2 letture su Utente per verificare l'esistenza del mittente e del destinatario e recuperare il recapito preferito di quest'ultimo; 1 lettura su Annuncio per verificarne l'esistenza e controllare che almeno uno dei due partecipanti coincida con l'autore; 1 scrittura su Messaggio_Privato. Se l'annuncio è già venduto, vengono inoltre esaminati i messaggi precedenti per verificare che la conversazione tra i due utenti sia stata avviata prima della vendita.
- **Costo unitario:** $3R + 2(1W) = 5$ accessi
- **Costo complessivo:** $5 \times 8.000 = 40.000$ accessi/giorno

Il controllo aggiuntivo sulle conversazioni precedenti si applica soltanto ai messaggi relativi ad annunci già venduti e non viene quindi incluso nel costo ordinario.

OP8 – Iscrizione o disiscrizione dal monitoraggio di un annuncio (Segui)

- **Frequenza:** 2.500/giorno
- **Accessi logici:** : nel caso di iscrizione, 1 lettura su Annuncio per verificare che lo Stato sia "InVendita" e che l'utente non coincida con l'Autore; 1 lettura su Utente per verificarne l'esistenza; 1 scrittura su Segui per inserire la coppia formata dall'utente e dall'annuncio. Nel caso di disiscrizione viene eliminata da Segui la coppia corrispondente. Il costo viene stimato considerando il caso più oneroso dell'iscrizione.
- **Costo unitario:** $2R + 2(1W) = 4$ accessi
- **Costo complessivo:** $4 \times 2.500 = 10.000$ accessi/giorno

OP9 – Modifica delle informazioni di un annuncio attivo con individuazione dei follower

- **Frequenza:** 300/giorno
- **Accessi logici:** 1 lettura su Annuncio per verificarne l'esistenza, controllare che lo Stato sia "InVendita" e accertare che l'utente richiedente coincida con l'Autore; 1 lettura su Categoria per verificare l'esistenza della categoria assegnata; 1 scrittura su Annuncio per aggiornare Titolo, DescrizioneArticolo, Prezzo e Categoria; 4 letture per recuperare mediamente 2 follower e i rispettivi recapiti preferiti, di cui 2 su Segui e 2 su Utente.
- **Costo unitario:** $6R + 2(1W) = 8$ accessi
- **Costo complessivo:** $8 \times 300 = 2.400$ accessi / giorno

OP10 – Aggiornamento dello stato di un annuncio a "Venduto" con individuazione dei follower

- **Frequenza:** 400/giorno

- **Accessi logici:** 1 lettura su Annuncio per verificarne l'esistenza, controllare che sia ancora nello stato "InVendita" e accertare che l'utente richiedente coincida con l'Autore; 1 scrittura su Annuncio per impostare Stato = "Venduto"; 4 letture per recuperare mediamente 2 follower e i rispettivi recapiti preferiti, di cui 2 su Segui e 2 su Utente.
- **Costo unitario:** $5R + 2(1W) = 7$ accessi
- **Costo complessivo:** $7 \cdot 400 = 2.800$ accessi/giorno

OP11 – Creazione e collocazione di una nuova categoria

- **Frequenza:** 2/mese
- **Accessi logici:** 1 lettura su Utente per verificare che il creatore appartenga alla specializzazione Gestore; 1 lettura su Categoria per verificare l'esistenza dell'eventuale categoria padre; 1 ulteriore lettura su Categoria per controllare che non esista già una categoria con lo stesso nome e la stessa categoria padre; 1 scrittura su Categoria.
- **Costo unitario:** $3R + 2(1W) = 5$ accessi
- **Costo complessivo:** $5 \cdot 2 = 10$ accessi/mese

OP12 – Generazione del report statistico aggregato per utente

- **Frequenza:** 1/settimana
- **Accessi logici:** 1 lettura su Utente per verificare che il soggetto richiedente appartenga alla specializzazione Gestore; operazione di sola lettura che esamina 50.000 record di Utente e 200.000 record di Annuncio. Gli annunci vengono raggruppati per autore, contando per ciascun utente il numero totale di annunci pubblicati e quelli aventi Stato = "Venduto". Se un utente non ha pubblicato alcun annuncio, la percentuale viene posta uguale a 0.
- **Costo unitario:** $250.001R = 250.001$ accessi
- **Costo complessivo:** $250.001 \cdot 1 = 250.001$ accessi/settimana

OP13 –Recupero dei dati necessari all'autenticazione e alla verifica del profilo

- **Frequenza:** 30.000/giorno
- **Accessi logici:** 1 lettura su Utente, identificato mediante Username, per recuperare i dati necessari alla verifica delle credenziali e determinare se il soggetto dispone del profilo di Utente registrato oppure di Gestore.
- **Costo unitario:** $1R = 1$ accesso
- **Costo complessivo:** $1 \cdot 30.000 = 30.000$ accessi/giorno

OP14 – Visualizzazione degli annunci pubblicati da un utente

- **Frequenza:** 5.000/giorno
- **Accessi logici:** mediamente 4 letture su Annuncio per recuperare gli annunci pubblicati dall'utente richiesto; mediamente 4 letture su Categoria per recuperare il nome della categoria associata a ciascun annuncio.

- Costo unitario: $8R = 8$ accessi
- Costo complessivo: $8 \cdot 5.000 = 40.000$ accessi/giorno

OP15 – Visualizzazione della conversazione privata tra due utenti relativa a un annuncio

- Frequenza: 15.000/giorno
- Accessi logici: 1 lettura su Annuncio per verificare che almeno uno dei partecipanti coincida con l'Autore; mediamente 30 letture su Messaggio_Privato per recuperare i messaggi scambiati nelle due direzioni tra il venditore e l'altro utente relativamente allo specifico annuncio. I messaggi vengono ordinati in base a DataOra e CodiceMessaggio per ricostruire cronologicamente la conversazione.
- Costo unitario medio: $31R = 31$ accessi
- Costo complessivo medio: $31 \cdot 15.000 = 465.000$ accessi/giorno

Ristrutturazione dello schema E-R

Analisi delle Ridondanze

È stata valutata l'introduzione in Utente degli attributi ridondanti NumAnnunciTotali e NumAnnunciVenduti, allo scopo di velocizzare la generazione del report statistico OP12.

- **Valutazione delle prestazioni:** senza ridondanza, OP12 richiede circa 250.000 letture, corrispondenti all'esame di 50.000 utenti e 200.000 annunci. Con i due contatori, il report richiederebbe soltanto 50.000 letture su Utente. La ridondanza comporterebbe però una scrittura aggiuntiva su Utente in occasione della pubblicazione di un annuncio (OP2) e dell'aggiornamento dello stato a "Venduto" (OP10).
- **Occupazione di memoria:** la memorizzazione di due valori interi per 50.000 utenti richiederebbe circa 400 KB e avrebbe quindi un impatto trascurabile.
- **Decisione:** si sceglie di non introdurre la ridondanza. Nonostante il vantaggio prestazionale per OP12, il report viene generato soltanto una volta alla settimana e può essere eseguito in un momento di basso carico. Si preferisce evitare l'aggiornamento e la sincronizzazione dei contatori durante operazioni quotidiane molto più frequenti, garantendo che il report sia sempre calcolato direttamente sui dati effettivi degli annunci.

Eliminazione delle generalizzazioni

Nello schema concettuale è presente una gerarchia di generalizzazione: l'entità genitore Utente e la sotto-entità figlia Gestore. Si tratta di una generalizzazione parziale (non tutti gli utenti sono gestori).

Sono state analizzate le tre possibili strategie di eliminazione previste dalla teoria:

- **Accorpamento del genitore nei figli:** Non è applicabile, poiché la generalizzazione è parziale; adottando questa soluzione si perderebbero tutti gli utenti normali che non sono gestori.
- **Mantenimento delle entità con relazione 1 a 1:** Creerebbe una tabella separata per soli 10 gestori su 50.000 utenti. Questa scelta costringerebbe a fare un'operazione di JOIN in più ogni volta che si deve verificare se un utente è un gestore, risultando poco efficiente.
- **Accorpamento del figlio nel genitore (Scelta adottata):** L'entità figlia Gestore viene eliminata e integrata direttamente dentro l'entità Utente.

Modifiche applicate allo schema:

- Si aggiunge all'entità Utente un attributo booleano chiamato IsGestore (che assume valore *vero* se l'utente è un amministratore, *false* altrimenti).
- La relazione Crea viene collegata direttamente tra Utente e Categoria, con cardinalità (0,N) lato utente.
- A seguito dell'accorpamento della specializzazione Gestore nell'entità Utente, RV5 viene realizzata imponendo che l'utente indicato come GestoreCreatore di una Categoria abbia IsGestore = TRUE.

Trasformazione degli Attributi Composti

Nel modello relazionale non sono ammessi attributi strutturati o composti; tutti gli attributi devono essere atomici. Gli attributi composti presenti nell'entità Utente sono stati trasformati come segue:

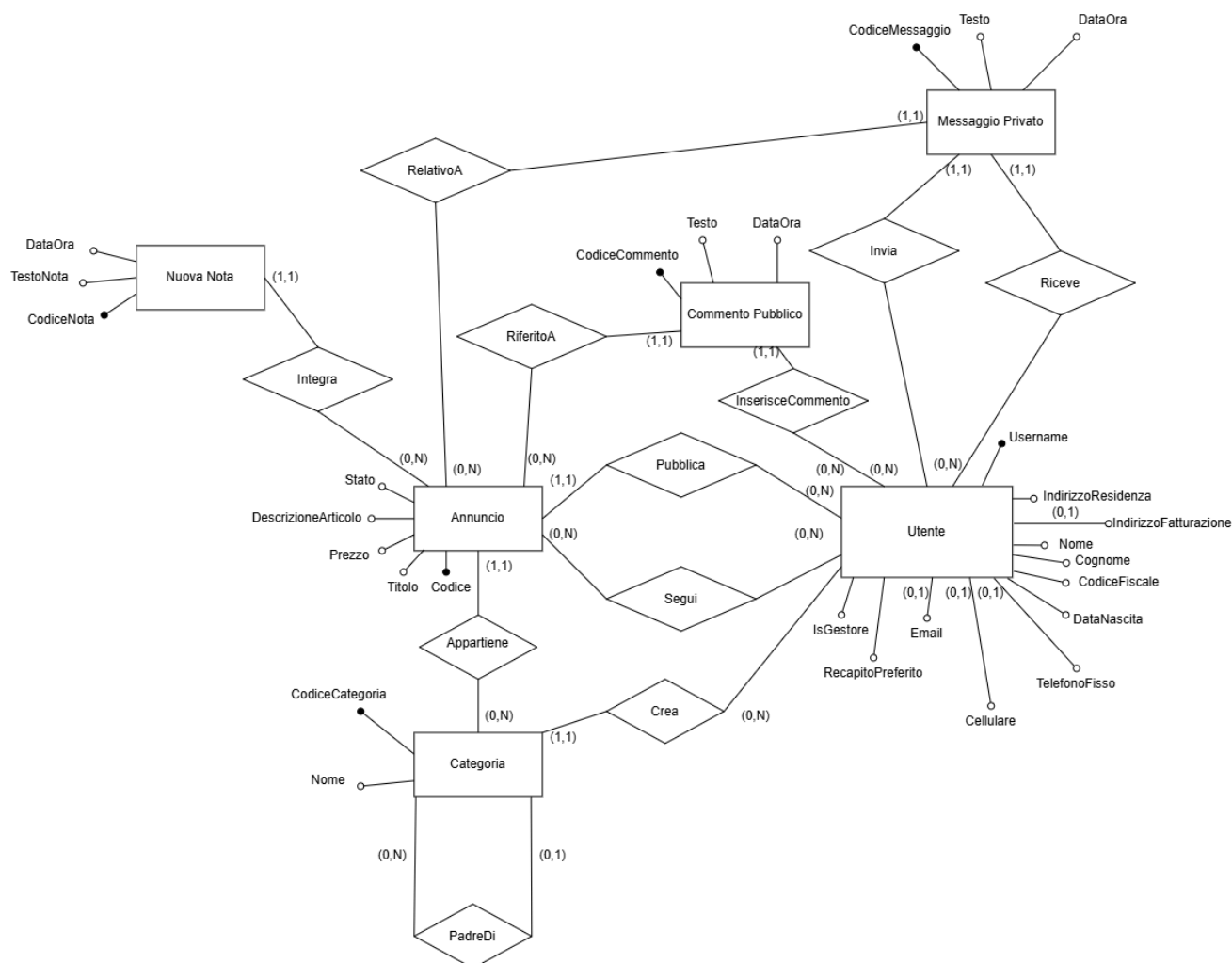
- **Attributo Dati anagrafici:** È stato eliminato e i suoi componenti (Nome, Cognome, CodiceFiscale, DataNascita) sono stati collegati direttamente come attributi semplici all'entità Utente.
- **Attributo Recapito:** È stato eliminato e i tre canali di contatto (Email, Cellulare, TelefonoFisso), aventi cardinalità massima pari a 1 per ciascun tipo, sono stati inseriti direttamente come attributi opzionali nell'entità Utente.

Scelta degli identificatori primari

Per ciascuna entità dello schema ristrutturato è stato scelto un identificatore principale semplice, composto da un solo attributo e privo di valori nulli. Quando non era disponibile un identificatore naturale adeguato, è stato introdotto un codice numerico surrogato:

- **Utente → Username:** identificatore naturale di tipo stringa, obbligatorio e univoco fin dalla fase di registrazione. Si preferisce Username rispetto a CodiceFiscale poiché rappresenta l'identificativo scelto dall'utente ed è utilizzato per riferirsi a esso nelle operazioni della piattaforma.
- **Annuncio → Codice:** Identificatore surrogato numerico (intero). È preferito a combinazioni di attributi descrittivi (come autore e data) perché garantisce compattezza e velocità nel collegare commenti, messaggi e note.

- **Categoria** → **CodiceCategoria**: Identificatore surrogato numerico (intero), ideale e compatto per supportare in modo efficiente la relazione gerarchica ricorsiva PadreDi.
- **Commento Pubblico** → **CodiceCommento**: Identificatore numerico univoco generato automaticamente.
- **Messaggio Privato** → **CodiceMessaggio**: Identificatore numerico univoco generato automaticamente.
- **Nuova Nota** → **CodiceNota**: Identificatore numerico univoco generato automaticamente.



Trasformazione di attributi e identificatori

Lo schema non necessita di ulteriori trasformazioni strutturali:

- **Tutti gli attributi sono atomici** e a cardinalità singola.
- **Tutte le entità possiedono un identificatore principale interno e semplice**, garantendo una mappatura diretta verso il modello logico-relazionale.

Traduzione di entità e associazioni

Utente (Username, IndirizzoResidenza, IndirizzoFatturazione*, Nome, Cognome, CodiceFiscale, DataNascita, TelefonoFisso*, Cellulare*, Email*, RecapitoPreferito, IsGestore);

Categoria (CodiceCategoria, Nome, **CategoriaPadre***, **GestoreCreatore**);

Annuncio (Codice, Titolo, DescrizioneArticolo, Prezzo, Stato, **Categoria**, **Autore**);

Commento_Pubblico (CodiceCommento, Testo, DataOra, **Annuncio**, **Autore**);

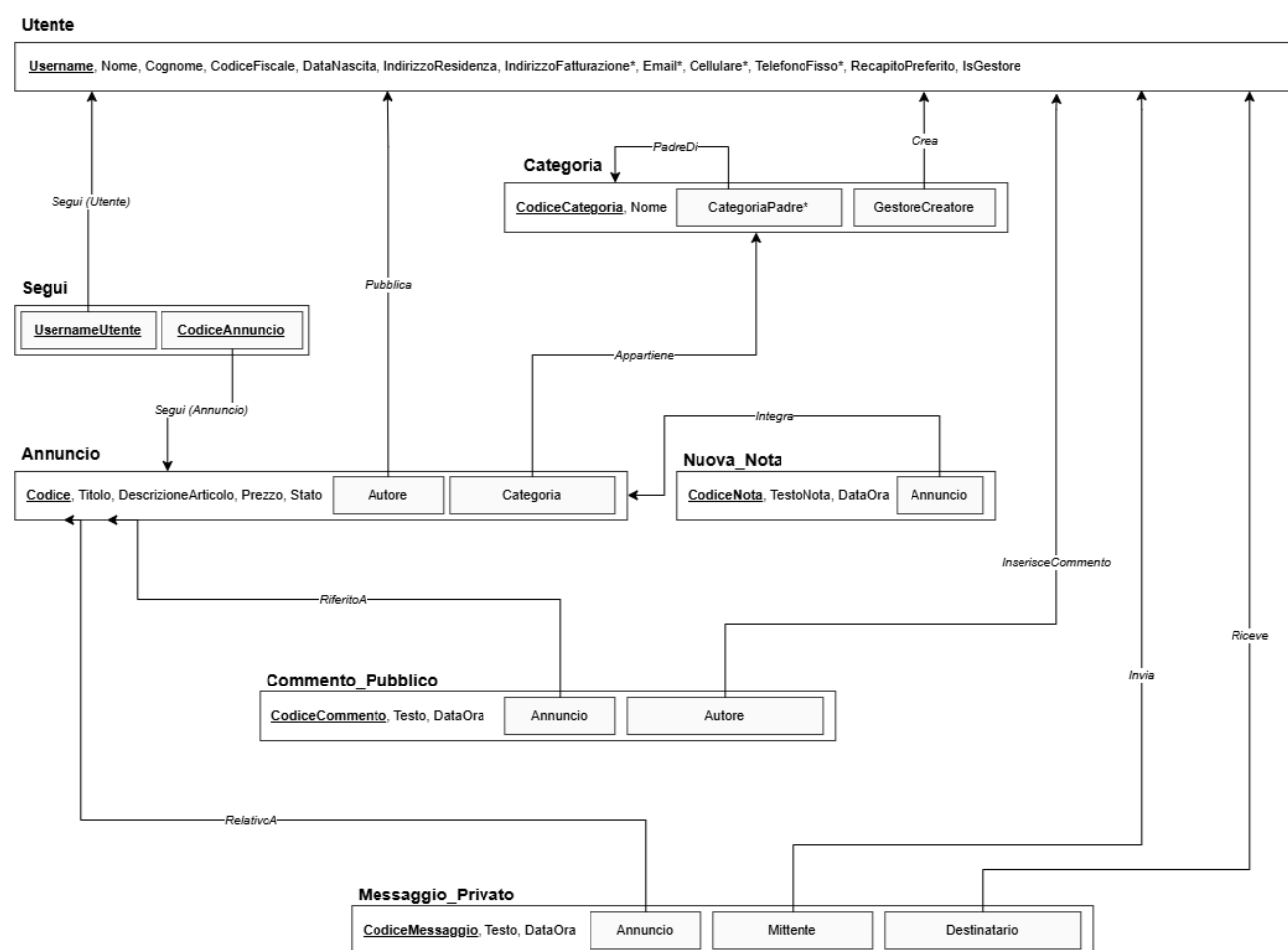
Messaggio_Privato (CodiceMessaggio, Testo, DataOra, **Annuncio**, **Destinatario**, **Mittente**);

Nuova_Nota (CodiceNota, TestoNota, DataOra, **Annuncio**);

Segui (**UsernameUtente**, **CodiceAnnuncio**);

Note di Progettazione Logica

- **Chiavi Primarie e Foreign Key:** La chiave primaria di ciascuna tabella è sottolineata, mentre le chiavi esterne (FK) sono riportate in verde.
- **Relazione N a N (Segui):** L'associazione molti a molti Segui è stata tradotta in una tabella ponte autonoma la cui chiave primaria è composita, formata dalle due foreign key (**UsernameUtente**, **CodiceAnnuncio**) che referenziano rispettivamente le tabelle Utente e Annuncio.
- **Gerarchia Ricorsiva (Categoria):** La relazione gerarchica PadreDi è tradotta tramite la foreign key CategoriaPadre che referenzia la chiave primaria della tabella stessa (Categoria), consentendo valori nulli per le categorie radice.
- **Accorpamento delle Relazioni 1 a N:** le associazioni uno a molti Pubblica, Appartiene, Crea, Integra, RiferitoA, InserisceCommento, Invia, Riceve e RelativoA sono state tradotte importando la chiave primaria dell'entità posta sul lato 1 come chiave esterna nella relazione corrispondente al lato N.



Normalizzazione del modello relazionale

Lo schema relazionale ottenuto dalla ristrutturazione rispetta pienamente la **Forma Normale di Boyce-Codd (BCNF)** e, di conseguenza, anche la **Terza Forma Normale (3NF)**. Non sono necessarie ulteriori scomposizioni o denormalizzazioni.

Analisi delle Dipendenze Funzionali Tabella per Tabella

• UTENTE

Dipendenze funzionali non banali:

- Username → Nome, Cognome, CodiceFiscale, DataNascita, IndirizzoResidenza, IndirizzoFatturazione, Email, Cellulare, TelefonoFisso, RecapitoPreferito, IsGestore
- CodiceFiscale → Username, Nome, Cognome, DataNascita, IndirizzoResidenza, IndirizzoFatturazione, Email, Cellulare, TelefonoFisso, RecapitoPreferito, IsGestore

Verifica: Tutte le dipendenze funzionali hanno come determinante una superchiave (Username è la PK, CodiceFiscale è una chiave candidata alternativa). Non esistono dipendenze transitive né parziali. -> In BCNF

- **CATEGORIA**

Dipendenze funzionali:

- CodiceCategoria->Nome, CategoriaPadre, GestoreCreatore
- {Nome, CategoriaPadre}-> CodiceCategoria, GestoreCreatore

Verifica: La seconda dipendenza deriva da RV9, che impedisce l'esistenza di due categorie con lo stesso nome e la stessa categoria padre, comprese le categorie radice. CodiceCategoria è la chiave primaria, mentre la coppia {Nome, CategoriaPadre} costituisce una chiave candidata alternativa. Entrambi i determinanti sono quindi superchiavi e la relazione è in **BCNF**.

- **ANNUNCIO**

Dipendenze funzionali:

- Codice->Titolo, DescrizioneArticolo, Prezzo, Stato, Autore, Categoria

Verifica: Ogni attributo non chiave dipende esclusivamente dall'intero codice identificativo dell'annuncio. → **In BCNF**.

- **COMMENTO_PUBBLICO**

Dipendenze funzionali:

- CodiceCommento->Testo, DataOra, Annuncio, Autore

Verifica: Il determinante coincide con la chiave primaria. → **In BCNF**.

- **MESSAGGIO_PRIVATO**

Dipendenze funzionali:

- CodiceMessaggio->Testo, DataOra, Annuncio, Mittente, Destinatario

Verifica: Il determinante coincide con la chiave primaria. → **In BCNF**.

- **NUOVA_NOTA**

Dipendenze funzionali:

- CodiceNota->TestoNota, DataOra, Annuncio

Verifica: Il determinante coincide con la chiave primaria. → **In BCNF**.

- **SEGUI**

Dipendenze funzionali: Solo dipendenze banali del tipo

- $\{\text{UsernameUtente}, \text{CodiceAnnuncio}\} \rightarrow \text{UsernameUtente}$

Verifica: Non ci sono attributi non chiave; la totalità degli attributi forma la chiave primaria composita. → **In BCNF**.

Sintesi della Verifica

Tutte le relazioni dello schema soddisfano la condizione di BCNF: per ogni dipendenza funzionale $X \rightarrow Y$, X è una **superchiave**.

5. Progettazione fisica

Utenti e privilegi

All'interno dell'applicazione sono previsti tre profili: Ospite, Utente registrato e Gestore del sistema. A tali profili corrispondono rispettivamente i ruoli MySQL “ruolo_accesso”, “ruolo_utente” e “ruolo_gestore”.

I ruoli non ricevono privilegi diretti sulle tabelle o sulle viste. L'accesso ai dati avviene esclusivamente mediante le stored procedure per le quali è stato concesso il privilegio EXECUTE. In questo modo gli account utilizzati dall'applicazione non possono aggirare i controlli previsti eseguendo direttamente istruzioni SELECT, INSERT, UPDATE o DELETE.

Le stored procedure verificano inoltre la proprietà degli annunci, il loro stato, la riservatezza delle conversazioni e, per le operazioni amministrative, l'appartenenza al profilo Gestore.

1. Ospite / Utente non autenticato (ruolo_accesso)

- **Ruolo:** utente che non ha ancora effettuato l'accesso alla piattaforma
- **Operazioni consentite:**
 - **OP1:** registrazione di un nuovo utente;
 - **OP13:** recupero delle informazioni necessarie alla verifica delle credenziali e del profilo.
- **Limitazioni:** non può consultare né modificare i dati della bacheca prima che la propria identità sia stata verificata.

La registrazione inserisce il nuovo soggetto nella tabella Utente impostando “IsGestore” = FALSE, in conformità con RV8. In questo modo un nuovo utente non può attribuirsi autonomamente il profilo Gestore.

Al ruolo “ruolo_accesso” viene concesso il privilegio EXECUTE esclusivamente sulle procedure “sp_RegistraUtente” e “sp_RecuperaProfiloAccesso”.

- **Giustificazione:** prima dell'autenticazione il soggetto deve poter soltanto registrarsi o fornire le credenziali necessarie alla propria identificazione.

2. Utente Registrato (ruolo_utente)

- **Ruolo:** Soggetto registrato e autenticato che utilizza le funzionalità ordinarie della piattaforma.
- **Operazioni consentite:**
 - OP2: pubblicazione di un nuovo annuncio;
 - OP3: pubblicazione di una nota integrativa e individuazione dei follower;
 - OP4: ricerca degli annunci per categoria e sottocategorie;
 - OP5: visualizzazione della scheda di dettaglio di un annuncio;
 - OP6: inserimento di un commento pubblico;
 - OP7: invio di un messaggio privato;
 - OP8: iscrizione e disiscrizione dal monitoraggio di un annuncio;
 - OP9: modifica delle informazioni di un proprio annuncio attivo;
 - OP10: aggiornamento di un proprio annuncio allo stato “Venduto”;
 - OP14: visualizzazione dei propri annunci;
 - OP15: visualizzazione di una conversazione privata relativa a un annuncio.

Al ruolo “ruolo_utente” viene concesso il privilegio EXECUTE esclusivamente sulle stored procedure necessarie all’esecuzione di tali operazioni.

- **Limitazioni:** un utente può aggiungere note, modificare le informazioni o contrassegnare come venduti esclusivamente gli annunci di cui è autore e che si trovano nello stato “InVendita”. Non può seguire il proprio annuncio né iniziare a seguire un annuncio già venduto. Dopo la vendita non può inserire nuovi commenti pubblici né avviare nuove conversazioni private, mentre può proseguire quelle già esistenti. Può visualizzare soltanto le conversazioni private delle quali è partecipante. Non può creare categorie né generare il report statistico complessivo.
- **Giustificazione:** questo profilo comprende le funzionalità ordinarie necessarie alla pubblicazione degli annunci e all’interazione con gli altri utenti, senza concedere funzionalità amministrative.

3. Gestore del Sistema (ruolo_gestore)

- **Ruolo:** utente registrato avente “IsGestore” = TRUE, autorizzato a creare la struttura delle categorie e a generare il report statistico complessivo.

- **Operazioni consentite:**

- tutte le operazioni attribuite al ruolo “ruolo_utente”;
- **OP11:** creazione e collocazione di una nuova categoria nell’albero gerarchico;
- **OP12:** generazione del report contenente, per ciascun utente, il numero totale di annunci, il numero di annunci venduti e la relativa percentuale.

Il ruolo “ruolo_gestore” eredita i privilegi di “ruolo_utente” e riceve inoltre il privilegio EXECUTE sulle procedure “sp_CreaCategoria” e “sp_GeneraReportUtenti”. La procedura di generazione del report controlla comunque che lo username ricevuto appartenga a un utente avente “IsGestore” = TRUE, applicando RV12 come ulteriore livello di verifica.

- **Giustificazione:** il Gestore necessita di funzionalità aggiuntive per organizzare la classificazione degli annunci e consultare le statistiche complessive della piattaforma.

Autenticazione e verifica del profilo

La password non fa parte dei requisiti originari del minimondo e non viene quindi rappresentata nello schema concettuale e logico. Nella progettazione fisica viene introdotto nella tabella Utente l’attributo tecnico “PasswordHash”, necessario per l’autenticazione.

“PasswordHash” contiene il risultato dell’algoritmo PBKDF2 con HMAC-SHA-256 applicato alla password insieme a un salt casuale. La password originale non viene mai memorizzata nel database.

Durante l’accesso, Java utilizza l’account associato a “ruolo_accesso” per recuperare, mediante OP13, “PasswordHash” e “IsGestore”. La password fornita viene verificata dalla classe “PasswordHasher”. Se la verifica fallisce, l’accesso viene negato.

Se la verifica ha esito positivo, Java determina il profilo dell’utente:

- se “IsGestore” = FALSE, utilizza l’account associato a ruolo_utente;
- se “IsGestore” = TRUE, utilizza l’account associato a ruolo_gestore.

Prima di aprire la sessione applicativa, la procedura sp_VerificaProfiloSessione controlla che l’account MySQL utilizzato sia coerente con il valore di “IsGestore”. Lo stesso controllo viene eseguito all’apertura delle successive connessioni operative. In questo modo non è sufficiente modificare il ruolo conservato da Java per ottenere privilegi amministrativi.

Lo username autenticato viene conservato nella sessione applicativa e passato automaticamente alle procedure che devono verificare la proprietà degli annunci o la partecipazione alle conversazioni.

Gli account tecnici e la loro associazione ai ruoli sono definiti nello script “security.sql”. Le relative password non sono memorizzate nel codice sorgente, ma vengono fornite all’applicazione mediante variabili d’ambiente.

Strutture di memorizzazione

Tabella Utente		
Colonna	Tipo di dato	Attributi ²
Username	VARCHAR(30)	PK, NN
PasswordHash	VARCHAR(255)	NN
Nome	VARCHAR(50)	NN
Cognome	VARCHAR(50)	NN
CodiceFiscale	CHAR(16)	NN, UQ
DataNascita	DATE	NN
IndirizzoResidenza	VARCHAR(100)	NN
IndirizzoFatturazione	VARCHAR(100)	
Email	VARCHAR(100)	
Cellulare	VARCHAR(20)	
TelefonoFisso	VARCHAR(20)	
RecapitoPreferito	VARCHAR(15)	NN
IsGestore	BOOLEAN	NN, DEFAULT FALSE

Tabella Categoria		
Colonna	Tipo di dato	Attributi
CodiceCategoria	INT	PK, NN, UN, AI
Nome	VARCHAR(50)	NN
CategoriaPadre	INT	UN
GestoreCreatore	VARCHAR(30)	NN

Tabella Annuncio		
Colonna	Tipo di dato	Attributi
Codice	INT	PK, NN, UN, AI
Titolo	VARCHAR(100)	NN
DescrizioneArticolo	TEXT	NN
Prezzo	DECIMAL(10,2)	NN,UN

² PK = primary key, NN = not null, UQ = unique, UN = unsigned, AI = auto increment. È ovviamente possibile specificare più di un attributo per ciascuna colonna.

Stato	VARCHAR(20)	NN, dominio {InVendita, Venduto}, "InVendita" DEFAULT
Autore	VARCHAR(30)	NN
Categoria	INT	NN, UN

Tabella Nuova_Nota		
Colonna	Tipo di dato	Attributi
CodiceNota	INT	PK, NN, UN, AI
TestoNota	TEXT	NN
DataOra	DATETIME	NN
Annuncio	INT	NN, UN

Tabella Commento_Pubblico		
Colonna	Tipo di dato	Attributi
CodiceCommento	INT	PK, NN, UN, AI
Testo	TEXT	NN
DataOra	DATETIME	NN
Annuncio	INT	NN, UN
Autore	VARCHAR(30)	NN

Tabella Messaggio_Privato		
Colonna	Tipo di dato	Attributi
CodiceMessaggio	INT	PK, NN, UN, AI
Testo	TEXT	NN
DataOra	DATETIME	NN
Annuncio	INT	NN, UN
Mittente	VARCHAR(30)	NN
Destinatario	VARCHAR(30)	NN

Tabella Segui		
Colonna	Tipo di dato	Attributi
UsernameUtente	VARCHAR(30)	PK, NN
CodiceAnnuncio	INT	PK, NN, UN

Indici

Tabella <Utente>	
Indice <Primary>	Tipo ³ :
Username	<PR>
Indice <uq Utente codice fiscale>	
CodiceFiscale	<UQ>

Spiegazione: l'indice Primary su Username deriva dalla definizione della chiave primaria e non deve essere creato separatamente. Esso supporta OP13, relativa all'autenticazione e al recupero del profilo dell'utente, caratterizzata da una frequenza di 30.000 esecuzioni giornaliere. L'indice viene inoltre utilizzato dal DBMS per verificare i vincoli di integrità referenziale delle tabelle che fanno riferimento a Utente, come avviene durante l'iscrizione a un annuncio in OP8. Il vincolo UNIQUE su CodiceFiscale genera invece l'indice univoco "uq_Utente_codice_fiscale", necessario a impedire la registrazione di più utenti con lo stesso codice fiscale. Non sono necessari ulteriori indici sugli altri attributi.

Tabella <Categoria>	
Indice <Primary>	Tipo:
CodiceCategoria	<PR>
Indice <idx_Categoria_padre>	
CategoriaPadre	<IDX>

Spiegazione: L'indice su CategoriaPadre serve ad attraversare la gerarchia PadreDi senza scansionare l'intera tabella ogni volta. Viene sfruttato da OP4 (40.000/giorno) quando si cercano le sottocategorie di una categoria data.

Tabella <Annuncio>	
Indice <Primary>	Tipo:
Codice	<PR>
Indice <idx_Annuncio_categoria_stato>	
Categoria, Stato	<IDX>
Indice <idx_Annuncio_autore>	
Autore	<IDX>

Spiegazione: L'indice composito (Categoria, Stato) supporta OP4, l'operazione più frequente del sistema, nella quale gli annunci vengono filtrati in base alla categoria e allo stato "InVendita".

³ IDX = index, UQ = unique, FT = full text, PR = primary.

L'indice su Autore supporta OP14, che recupera gli annunci pubblicati da uno specifico utente, ed è inoltre utile per l'aggregazione per autore richiesta da OP12.

Tabella <Commento_Pubblico>	
Indice <Primary>	Tipo:
CodiceCommento	<PR>
Indice <idx_Commento_annuncio>	
Annuncio	<IDX>

Tabella <Nuova_Nota>	
Indice <Primary>	Tipo:
CodiceNota	<PR>
Indice <idx_Nota_annuncio>	
Annuncio	<IDX>

Spiegazione indici per Commento_Pubblico e Nuova_Nota: Entrambe le tabelle vengono interrogate da OP5 (60.000/giorno) filtrando su Annuncio, non sulla propria chiave primaria: senza questo indice ogni apertura di una scheda annuncio richiederebbe uno scan su tabelle da 600.000 e 100.000 righe rispettivamente.

Tabella <Messaggio_Privato>	
Indice <Primary>	Tipo:
CodiceMessaggio	<PR>
Indice <idx_Messaggio_conversazione>	
Annuncio, Mittente, Destinatario	<IDX>
Indice <idx_Messaggio_data_ora>	
DataOra	<IDX>

Spiegazione: l'indice composto idx_Messaggio_conversazione sugli attributi (Annuncio, Mittente, Destinatario) supporta OP15, eseguita circa 15.000 volte al giorno. L'operazione recupera i messaggi relativi a uno specifico annuncio scambiati tra due determinati utenti, considerando entrambe le direzioni della comunicazione. I risultati ottenuti vengono successivamente uniti e ordinati in base a DataOra, così da ricostruire cronologicamente la conversazione.

L'indice idx_Messaggio_data_ora, definito sul solo attributo DataOra, supporta invece l'evento mensile ev_EliminaMessaggiObsoleti. L'evento ricerca ed elimina i messaggi privati risalenti a più di tre anni prima. Trattandosi di una ricerca per intervallo temporale, l'indice consente di individuare i messaggi interessati senza effettuare una scansione completa della tabella.

I due indici vengono mantenuti separati perché supportano operazioni differenti: il primo ottimizza il recupero delle conversazioni, mentre il secondo ottimizza la manutenzione periodica dei dati.

Tabella <Segui>	
Indice <Primary>	Tipo:
UsernameUtente, CodiceAnnuncio	<PR>
Indice <idx_Segui_annuncio>	
CodiceAnnuncio	<IDX>

Spiegazione: La chiave primaria composta (UsernameUtente, CodiceAnnuncio) impedisce che un utente segua più volte lo stesso annuncio e supporta OP8 nelle operazioni di iscrizione, verifica e disiscrizione. Poiché la chiave è ordinata a partire da UsernameUtente, non è invece adatta alla ricerca inversa di tutti gli utenti che seguono uno specifico annuncio. L'indice su CodiceAnnuncio consente di recuperare tutti gli utenti che seguono un determinato annuncio ed è utilizzato da OP3, OP9 e OP10, aventi una frequenza complessiva di 1.200 esecuzioni giornaliere, per individuare i destinatari delle notifiche. Senza tale indice, ogni modifica notificabile richiederebbe una scansione completa delle 400.000 righe della tabella Segui.

Oltre agli indici descritti, MySQL può creare automaticamente gli indici necessari a supportare i vincoli di chiave esterna. Tali indici non rappresentano ulteriori scelte progettuali di ottimizzazione, ma sono funzionali al controllo dell'integrità referenziale.

Trigger

Sono stati implementati 14 trigger di tipo BEFORE, utilizzati per impedire l'inserimento o la modifica di dati che violino le regole aziendali. I controlli vengono quindi eseguiti prima che la nuova riga sia memorizzata.

Quando viene rilevata una violazione, il trigger esegue SIGNAL SQLSTATE '45000', interrompendo l'istruzione e restituendo un messaggio che indica la regola non rispettata.

Alcuni vincoli semplici potrebbero essere espressi mediante vincoli CHECK; tuttavia, i controlli che richiedono la consultazione di altre righe o tabelle, come la verifica del ruolo del creatore di una categoria o l'assenza di cicli nella gerarchia, richiedono una logica più articolata. L'impiego dei trigger permette inoltre di applicare tali regole anche quando i dati vengono modificati al di fuori delle stored procedure dell'applicazione.

Per le regole che devono continuare a essere rispettate anche dopo l'inserimento iniziale sono stati definiti sia un trigger BEFORE INSERT, indicato dalla sigla BI, sia un trigger BEFORE UPDATE, indicato dalla sigla BU.

I trigger implementati sono i seguenti:

- “trg_Utente_BI_recapiti” e “trg_Utente_BU_recapiti” realizzano RV1, RV2, RV14, RV15, RV16, RV17 e, limitatamente agli attributi testuali di Utente, RV19. Verificano la presenza di almeno un recapito, la validità del recapito preferito, il formato del codice fiscale, dell'indirizzo e-mail e dei numeri telefonici, la validità della data di nascita e la valorizzazione dei campi testuali obbligatori.
- “trg_Categoria_BI_vincoli” e “trg_Categoria_BU_vincoli” realizzano RV5, RV9 e, per il nome della categoria, RV19. Verificano che il creatore appartenga alla specializzazione Gestore e impediscono l'esistenza di due categorie con lo stesso nome e la stessa categoria padre, comprese le categorie radice.
- “trg_Categoria_BU_vincoli” realizza inoltre RV3. Partendo dal nuovo valore di “CategoriaPadre”, risale iterativamente la gerarchia; se incontra la categoria che si sta modificando, l'aggiornamento viene rifiutato perché produrrebbe un ciclo. Il controllo è necessario soltanto in aggiornamento, poiché una nuova categoria, identificata mediante un codice auto-incrementale, può riferirsi esclusivamente a categorie già esistenti.
- “trg_Segui_BI_autore” e “trg_Segui_BU_autore” realizzano RV10 e RV13. Impediscono all'autore di seguire il proprio annuncio e consentono l'iscrizione esclusivamente agli annunci ancora in vendita. La disiscrizione non viene ostacolata, poiché consiste nell'eliminazione di una riga e non attiva questi trigger.
- “trg_Annuncio_BI_stato” e “trg_Annuncio_BU_stato” realizzano RV7, RV18 e, per titolo e descrizione, RV19. Verificano che lo stato appartenga all'insieme {InVendita, Venduto}, che il prezzo sia maggiore di zero e che i campi testuali obbligatori non siano vuoti. Il valore iniziale “InVendita” è assegnato durante la pubblicazione ed è previsto anche come valore predefinito della colonna “Stato”.
- “trg_Messaggio_BI_vincoli” e “trg_Messaggio_BU_vincoli” realizzano RV4, RV6 e, per il testo del messaggio, RV19. Impediscono che mittente e destinatario coincidano e verificano che almeno uno dei due partecipanti sia l'autore dell'annuncio.
- “trg_Messaggio_BI_vincoli” realizza inoltre la parte di RV20 relativa alle conversazioni private. Se l'annuncio è stato venduto, un nuovo messaggio può essere inserito soltanto

quando tra i due utenti esiste già una conversazione relativa a quell’annuncio. In questo modo non è possibile avviare nuove trattative, ma le conversazioni precedenti possono continuare.

- “trg_Nota_BI_testo” e “trg_Nota_BU_testo” realizzano RV19 per Nuova_Nota, impedendo l’inserimento o la memorizzazione di un testo nullo, vuoto o composto esclusivamente da spazi.
- “trg_Commento_BI_testo” e “trg_Commento_BU_testo” realizzano RV19 per Commento_Pubblico. Il trigger di inserimento realizza inoltre RV20, impedendo la pubblicazione di nuovi commenti relativi ad annunci già venduti.

RV8 non viene realizzata mediante trigger perché riguarda specificamente l’operazione di registrazione. La stored procedure “sp_RegistraUtente” assegna sempre FALSE a “IsGestore”, impedendo al nuovo utente di attribuirsi autonomamente il profilo di Gestore.

RV11 viene controllata dalle stored procedure che pubblicano note, modificano gli annunci e ne aggiornano lo stato, poiché richiede di confrontare l’utente che richiede l’operazione con l’autore dell’annuncio. RV12 viene invece applicata mediante la stored procedure del report e il sistema di ruoli e privilegi del DBMS.

L’invio delle notifiche non viene effettuato mediante trigger. I trigger operano all’interno delle transazioni del database e non devono eseguire attività esterne, come l’invio di e-mail o SMS. Le stored procedure recuperano i destinatari e i relativi recapiti, mentre la simulazione dell’invio viene eseguita dalla logica applicativa.

Codice SQL

```
USE BachecaAnnunci;

DELIMITER $$

DROP TRIGGER IF EXISTS trg_Utente_BI_recapiti $$
CREATE TRIGGER trg_Utente_BI_recapiti
BEFORE INSERT ON Utente
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Nome), '') IS NULL
        OR NULLIF(TRIM(NEW.Cognome), '') IS NULL
```

```
OR NULLIF(TRIM(NEW.IndirizzoResidenza), '') IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV19: i campi testuali obbligatori
non possono essere vuoti';
END IF;

IF NOT REGEXP_LIKE(NEW.CodiceFiscale, '[A-Z0-9]{16}$', 'c')
THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV14: codice fiscale non valido';
END IF;

IF NEW.Email IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.Email,
'^[[:space:]]@]+@[[:space:]]@+[.][^[:space:]]@+$', 'c') THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV15: formato email non valido';
END IF;

IF (NEW.Cellulare IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.Cellulare, '[+]?[0-9]{7,15}$',
'c'))
OR (NEW.TelefonoFisso IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.TelefonoFisso, '[+]?[0-9]{7,15}$',
'c')) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV16: formato telefonico non
valido';
END IF;

IF NEW.DataNascita > CURRENT_DATE() THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV17: la data di nascita non puo
essere futura';
```

```
END IF;

IF NULLIF(TRIM(NEW.Email), '') IS NULL
AND NULLIF(TRIM(NEW.Cellulare), '') IS NULL
AND NULLIF(TRIM(NEW.TelefonoFisso), '') IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV1: inserire almeno un recapito';
END IF;

IF NEW.RecapitoPreferito NOT IN ('Email', 'Cellulare',
'TelefonoFisso')
OR (NEW.RecapitoPreferito = 'Email'
    AND NULLIF(TRIM(NEW.Email), '') IS NULL)
OR (NEW.RecapitoPreferito = 'Cellulare'
    AND NULLIF(TRIM(NEW.Cellulare), '') IS NULL)
OR (NEW.RecapitoPreferito = 'TelefonoFisso'
    AND NULLIF(TRIM(NEW.TelefonoFisso), '') IS NULL) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV2: il recapito preferito deve
essere valorizzato';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Utente_BU_recapiti $$
CREATE TRIGGER trg_Utente_BU_recapiti
BEFORE UPDATE ON Utente
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Nome), '') IS NULL
    OR NULLIF(TRIM(NEW.Cognome), '') IS NULL
    OR NULLIF(TRIM(NEW.IndirizzoResidenza), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: i campi testuali obbligatori
non possono essere vuoti';
```

```
END IF;

IF NOT REGEXP_LIKE(NEW.CodiceFiscale, '[A-Z0-9]{16}$', 'c')
THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV14: codice fiscale non valido';
END IF;

IF NEW.Email IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.Email,
'^[^\[:space:]]@]+@[^\[:space:]]@]+[.][^\[:space:]]@]+$ ', 'c') THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV15: formato email non valido';
END IF;

IF (NEW.Cellulare IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.Cellulare, '[+]?[0-9]{7,15}$',
'c'))
OR (NEW.TelefonoFisso IS NOT NULL
    AND NOT REGEXP_LIKE(NEW.TelefonoFisso, '[+]?[0-9]{7,15}$',
'c')) THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV16: formato telefonico non
valido';
END IF;

IF NEW.DataNascita > CURRENT_DATE() THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV17: la data di nascita non puo
essere futura';
END IF;

IF NULLIF(TRIM(NEW.Email), '') IS NULL
    AND NULLIF(TRIM(NEW.Cellulare), '') IS NULL
```

```
AND NULLIF(TRIM(NEW.TelefonoFisso), '') IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV1: inserire almeno un recapito';
END IF;

IF NEW.RecapitoPreferito NOT IN ('Email', 'Cellulare',
'TelefonoFisso')
OR (NEW.RecapitoPreferito = 'Email'
    AND NULLIF(TRIM(NEW.Email), '') IS NULL)
OR (NEW.RecapitoPreferito = 'Cellulare'
    AND NULLIF(TRIM(NEW.Cellulare), '') IS NULL)
OR (NEW.RecapitoPreferito = 'TelefonoFisso'
    AND NULLIF(TRIM(NEW.TelefonoFisso), '') IS NULL) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV2: il recapito preferito deve
essere valorizzato';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Categoria_BI_vincoli $$
CREATE TRIGGER trg_Categoria_BI_vincoli
BEFORE INSERT ON Categoria
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Nome), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: il nome della categoria non
puo essere vuoto';
    END IF;

    IF NOT EXISTS (
        SELECT 1
        FROM Utente
        WHERE Username = NEW.GestoreCreatore
```

```
        AND IsGestore = TRUE
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV5: il creatore della categoria
deve essere un gestore';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM Categoria
        WHERE TRIM(Nome) = TRIM(NEW.Nome)
            AND CategoriaPadre <=> NEW.CategoriaPadre
    ) THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV9: categoria gia presente con quel
padre';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_Categoria_BU_vincoli $$
CREATE TRIGGER trg_Categoria_BU_vincoli
BEFORE UPDATE ON Categoria
FOR EACH ROW
BEGIN
    DECLARE v_antenato INT UNSIGNED;

    IF NULLIF(TRIM(NEW.Nome), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV19: il nome della categoria non
puo essere vuoto';
    END IF;

    IF NOT EXISTS (
        SELECT 1
```

```
FROM Utente
WHERE Username = NEW.GestoreCreatore
      AND IsGestore = TRUE
) THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV5: il creatore della categoria
deve essere un gestore';
END IF;

IF EXISTS (
    SELECT 1
    FROM Categoria
    WHERE TRIM(Nome) = TRIM(NEW.Nome)
          AND CategoriaPadre <=> NEW.CategoriaPadre
          AND CodiceCategoria <> OLD.CodiceCategoria
) THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV9: categoria gia presente con quel
padre';
END IF;

SET v_antenato = NEW.CategoriaPadre;

WHILE v_antenato IS NOT NULL DO
    IF v_antenato = NEW.CodiceCategoria THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV3: la gerarchia delle
categorie non puo contenere cicli';
    END IF;

    SET v_antenato = (
        SELECT CategoriaPadre
        FROM Categoria
        WHERE CodiceCategoria = v_antenato
```

```
);  
END WHILE;  
END $$  
  
DROP TRIGGER IF EXISTS trg_Segui_BI_autore $$  
CREATE TRIGGER trg_Segui_BI_autore  
BEFORE INSERT ON Segui  
FOR EACH ROW  
BEGIN  
    IF EXISTS (  
        SELECT 1  
        FROM Annuncio  
        WHERE Codice = NEW.CodiceAnnuncio  
        AND Autore = NEW.UsernameUtente  
    ) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'RV10: l autore non puo seguire il  
proprio annuncio';  
    END IF;  
  
    IF EXISTS (  
        SELECT 1  
        FROM Annuncio  
        WHERE Codice = NEW.CodiceAnnuncio  
        AND Stato <> 'InVendita'  
    ) THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'RV13: si possono seguire solo  
annunci in vendita';  
    END IF;  
END $$  
  
DROP TRIGGER IF EXISTS trg_Segui_BU_autore $$  
CREATE TRIGGER trg_Segui_BU_autore
```

```
BEFORE UPDATE ON Segui
FOR EACH ROW
BEGIN
    IF EXISTS (
        SELECT 1
        FROM Annuncio
        WHERE Codice = NEW.CodiceAnnuncio
        AND Autore = NEW.UsernameUtente
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV10: l autore non puo seguire il
proprio annuncio';
    END IF;

    IF EXISTS (
        SELECT 1
        FROM Annuncio
        WHERE Codice = NEW.CodiceAnnuncio
        AND Stato <> 'InVendita'
    ) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV13: si possono seguire solo
annunci in vendita';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_Annuncio_BI_stato $$
CREATE TRIGGER trg_Annuncio_BI_stato
BEFORE INSERT ON Annuncio
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Titolo), '') IS NULL
    OR NULLIF(TRIM(NEW.DescrizioneArticolo), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
```

```
        SET MESSAGE_TEXT = 'RV19: titolo e descrizione non
possono essere vuoti';
    END IF;

    IF NEW.Prezzo <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV18: il prezzo deve essere maggiore
di zero';
    END IF;

    IF NEW.Stato NOT IN ('InVendita', 'Venduto') THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV7: stato dell annuncio non
valido';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_Annuncio_BU_stato $$
CREATE TRIGGER trg_Annuncio_BU_stato
BEFORE UPDATE ON Annuncio
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Titolo), '') IS NULL
    OR NULLIF(TRIM(NEW.DescrizioneArticolo), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: titolo e descrizione non
possono essere vuoti';
    END IF;

    IF NEW.Prezzo <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV18: il prezzo deve essere maggiore
di zero';
    END IF;
```

```
IF NEW.Stato NOT IN ('InVendita', 'Venduto') THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV7: stato dell annuncio non
valido';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Messaggio_BI_vincoli $$
CREATE TRIGGER trg_Messaggio_BI_vincoli
BEFORE INSERT ON Messaggio_Privato
FOR EACH ROW
BEGIN
    DECLARE v_autore VARCHAR(30);
    DECLARE v_stato VARCHAR(10);
    DECLARE v_conversazione_esistente INT DEFAULT 0;

    IF NULLIF(TRIM(NEW.Testo), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV19: il testo del messaggio non puo
essere vuoto';
    END IF;

    IF NEW.Mittente = NEW.Destinatario THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV4: mittente e destinatario devono
essere distinti';
    END IF;

    SELECT Autore, Stato
    INTO v_autore, v_stato
    FROM Annuncio
    WHERE Codice = NEW.Annuncio;
```

```
IF v_stato <> 'InVendita' THEN
    SELECT COUNT(*)
    INTO v_conversazione_esistente
    FROM Messaggio_Privato
    WHERE Annuncio = NEW.Annuncio
        AND (
            (Mittente = NEW.Mittente
             AND Destinatario = NEW.Destinatario)
            OR (Mittente = NEW.Destinatario
                AND Destinatario = NEW.Mittente)
        );
END IF;

IF v_stato <> 'InVendita'
    AND v_conversazione_esistente = 0 THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV20: non si puo avviare una nuova
conversazione per un annuncio venduto';
END IF;

IF v_autore IS NOT NULL
    AND NEW.Mittente <> v_autore
    AND NEW.Destinatario <> v_autore THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV6: uno dei partecipanti deve
essere autore dell annuncio';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Messaggio_BU_vincoli $$
CREATE TRIGGER trg_Messaggio_BU_vincoli
BEFORE UPDATE ON Messaggio_Privato
FOR EACH ROW
BEGIN
```

```
DECLARE v_autore VARCHAR(30);

IF NULLIF(TRIM(NEW.Testo), '') IS NULL THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: il testo del messaggio non puo
essere vuoto';
END IF;

IF NEW.Mittente = NEW.Destinatarario THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV4: mittente e destinatario devono
essere distinti';
END IF;

SET v_autore = (
    SELECT Autore
    FROM Annuncio
    WHERE Codice = NEW.Annuncio
);

IF v_autore IS NOT NULL
    AND NEW.Mittente <> v_autore
    AND NEW.Destinatarario <> v_autore THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV6: uno dei partecipanti deve
essere autore dell annuncio';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Nota_BI_testo $$
CREATE TRIGGER trg_Nota_BI_testo
BEFORE INSERT ON Nuova_Nota
FOR EACH ROW
BEGIN
```

```
IF NULLIF(TRIM(NEW.TestoNota), '') IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'RV19: il testo della nota non puo
essere vuoto';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Nota_BU_testo $$
CREATE TRIGGER trg_Nota_BU_testo
BEFORE UPDATE ON Nuova_Nota
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.TestoNota), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: il testo della nota non puo
essere vuoto';
    END IF;
END $$

DROP TRIGGER IF EXISTS trg_Commento_BI_testo $$
CREATE TRIGGER trg_Commento_BI_testo
BEFORE INSERT ON Commento_Pubblico
FOR EACH ROW
BEGIN
    DECLARE v_stato VARCHAR(10);

    IF NULLIF(TRIM(NEW.Testo), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV19: il testo del commento non puo
essere vuoto';
    END IF;

    SELECT Stato
    INTO v_stato
```

```
FROM Annuncio
WHERE Codice = NEW.Annuncio;

IF v_stato <> 'InVendita' THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV20: non si puo interagire con un
annuncio venduto';
END IF;
END $$

DROP TRIGGER IF EXISTS trg_Commento_BU_testo $$
CREATE TRIGGER trg_Commento_BU_testo
BEFORE UPDATE ON Commento_Pubblico
FOR EACH ROW
BEGIN
    IF NULLIF(TRIM(NEW.Testo), '') IS NULL THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'RV19: il testo del commento non puo
essere vuoto';
    END IF;
END $$

DELIMITER ;
```

Eventi

Eliminazione periodica dei messaggi privati obsoleti

È stato implementato un evento pianificato per eliminare automaticamente i messaggi privati risalenti a più di tre anni prima. Tale politica di conservazione ha lo scopo di evitare la crescita indefinita della tabella `Messaggio_Privato`, mantenendo comunque disponibili le conversazioni per un periodo ritenuto adeguato.

Il periodo di conservazione è fissato a tre anni, mentre il controllo viene effettuato una volta al mese. La frequenza mensile è proporzionata alla durata del periodo considerato e limita il numero di esecuzioni non necessarie.

La ricerca dei messaggi obsoleti è supportata dall'indice "`idx_Messaggio_data_ora`", definito durante la progettazione fisica sull'attributo `DataOra` della tabella `Messaggio_Privato`.

L'evento viene istanziato mediante il seguente codice SQL:

```
DROP EVENT IF EXISTS ev_EliminaMessaggiObsoleti;

CREATE EVENT ev_EliminaMessaggiObsoleti
ON SCHEDULE EVERY 1 MONTH
STARTS CURRENT_TIMESTAMP + INTERVAL 1 MONTH
ON COMPLETION PRESERVE
ENABLE
DO
    DELETE FROM Messaggio_Privato
    WHERE DataOra < CURRENT_TIMESTAMP - INTERVAL 3 YEAR;
```

La clausola `ON SCHEDULE EVERY 1 MONTH` stabilisce la frequenza mensile dell'esecuzione. La clausola `STARTS` pianifica la prima esecuzione a un mese dalla configurazione, mentre `ON COMPLETION PRESERVE` mantiene l'evento registrato nel DBMS per le esecuzioni successive. La clausola `ENABLE` rende attiva la pianificazione.

Durante la configurazione del sistema viene inoltre abilitato il MySQL Event Scheduler:

```
SET GLOBAL event_scheduler = ON;
```

L'evento "ev_EliminaMessaggiObsoleti" viene creato una sola volta durante la configurazione del sistema e rimane registrato nel DBMS per le esecuzioni successive. Durante la stessa fase viene abilitato il MySQL Event Scheduler, necessario affinché gli eventi pianificati vengano eseguiti. Non vengono creati eventi temporanei durante l'esecuzione delle stored procedure.

Viste

Sono state definite tre viste, utilizzate per raccogliere in un'unica struttura dati provenienti da più attributi o tabelle e semplificare le interrogazioni eseguite dalle stored procedure.

Le viste utilizzate non sono materializzate: memorizzano la definizione dell'interrogazione, non una copia dei risultati. I dati restituiti riflettono quindi il contenuto corrente delle tabelle su cui sono definite.

Vista dei recapiti preferiti

```
-- Dati necessari per indirizzare le notifiche simulate.

CREATE OR REPLACE VIEW v_RecapitiPreferiti AS

SELECT

    Username,

    RecapitoPreferito,

    CASE RecapitoPreferito

        WHEN 'Email' THEN Email

        WHEN 'Cellulare' THEN Cellulare

        WHEN 'TelefonoFisso' THEN TelefonoFisso

    END AS Recapito

FROM Utente;
```

La vista “v_RecapitiPreferiti” associa a ogni utente il valore del recapito indicato come preferito. L’espressione CASE seleziona il contenuto di Email, Cellulare oppure TelefonoFisso in base al valore di RecapitoPreferito.

La vista evita di ripetere questa selezione nelle procedure che devono recuperare i destinatari delle notifiche simulate e restituisce direttamente lo username, il tipo di recapito e il relativo valore.

Vista di dettaglio degli annunci

```
-- Informazioni principali mostrate nella scheda di un annuncio.
CREATE OR REPLACE VIEW v_DettaglioAnnuncio AS
SELECT
    a.Codice,
    a.Titolo,
    a.DescrizioneArticolo,
    a.Prezzo,
    a.Stato,
    a.Autore,
    c.CodiceCategoria,
    c.Nome AS NomeCategoria,
    u.Nome AS NomeAutore,
    u.Cognome AS CognomeAutore,
    u.RecapitoPreferito,
    CASE u.RecapitoPreferito
        WHEN 'Email' THEN u.Email
        WHEN 'Cellulare' THEN u.Cellulare
        WHEN 'TelefonoFisso' THEN u.TelefonoFisso
    END AS RecapitoAutore
FROM Annuncio AS a
JOIN Categoria AS c
    ON c.CodiceCategoria = a.Categoria
JOIN Utente AS u
    ON u.Username = a.Autore;
```

La vista “v_DettaglioAnnuncio” riunisce le informazioni principali di un annuncio con il nome della categoria e i dati essenziali del suo autore. Le associazioni sono realizzate collegando Annuncio a Categoria mediante il codice della categoria e a Utente mediante lo username dell’autore.

Anche in questo caso il recapito da mostrare viene determinato tramite un’espressione CASE. La vista viene utilizzata per la scheda di dettaglio prevista da OP5 e come supporto alla visualizzazione degli annunci seguiti e degli annunci pubblicati dall’utente.

Vista delle statistiche degli utenti

```
-- Statistiche correnti utilizzate dal report riservato ai gestori.
CREATE OR REPLACE VIEW v_StatisticheUtenti AS
SELECT
    u.Username,
    COUNT(a.Codice) AS AnnunciTotali,
    COALESCE(SUM(a.Stato = 'Venduto'), 0) AS AnnunciVenduti,
    ROUND(
        COALESCE(
            100.0 * SUM(a.Stato = 'Venduto') /
            NULLIF(COUNT(a.Codice), 0),
            0
        ),
        2
    ) AS PercentualeVenduti
FROM Utente AS u
LEFT JOIN Annuncio AS a
    ON a.Autore = u.Username
GROUP BY u.Username;
```

La vista “v_StatisticheUtenti” realizza RD1 e calcola, per ciascun utente, il numero totale di annunci, il numero di annunci venduti e la relativa percentuale.

Il LEFT JOIN permette di includere anche gli utenti che non hanno pubblicato annunci. In questo caso NULLIF evita la divisione per zero, mentre COALESCE assegna alla percentuale il valore convenzionale zero. Il risultato viene infine arrotondato a due cifre decimali mediante ROUND.

La vista viene utilizzata da OP12 per generare il report statistico complessivo riservato ai gestori.

Stored Procedures e transazioni

Le operazioni dell'applicazione sono state implementate mediante stored procedure. Gli account utilizzati dall'applicazione non possiedono privilegi diretti sulle tabelle, ma possono eseguire esclusivamente le procedure autorizzate per il proprio ruolo. In questo modo la logica di accesso ai dati e i controlli più importanti rimangono centralizzati nel DBMS.

Tutte le procedure sono dichiarate con SQL SECURITY DEFINER: durante l'esecuzione utilizzano i privilegi dell'utente che le ha definite, mentre il chiamante deve possedere soltanto il privilegio EXECUTE.

Le procedure che effettuano esclusivamente interrogazioni sono dichiarate READS SQL DATA; quelle che inseriscono, modificano o eliminano dati sono invece dichiarate MODIFIES SQL DATA. Queste indicazioni descrivono il comportamento della procedura e rendono più chiaro il tipo di accesso effettuato.

Procedure di accesso e registrazione

- “sp_RegistraUtente” implementa OP1. Inserisce un nuovo utente, memorizza l'hash della password ricevuto dall'applicazione e assegna sempre FALSE a IsGestore, realizzando RV8.
- “sp_RecuperaProfiloAccesso” implementa OP13. Recupera PasswordHash e IsGestore a partire dallo username, fornendo all'applicazione i dati necessari per verificare le credenziali e determinare il profilo.
- “sp_VerificaProfiloSessione” è una procedura interna di sicurezza. Controlla che l'account tecnico con cui è stata aperta la connessione sia coerente con il profilo dell'utente autenticato. In questo modo un utente ordinario non può utilizzare una connessione riservata ai gestori.

Procedure relative agli annunci

- “sp_ElencoCategorie” restituisce le categorie disponibili ed è utilizzata come supporto durante la pubblicazione e la modifica di un annuncio.
- “sp_PubblicaAnnuncio” implementa OP2. Inserisce un nuovo annuncio associandolo all'autore e alla categoria scelta. Lo stato iniziale viene impostato a InVendita.

- “sp_RicercaAnnunciPerCategoria” implementa OP4. Utilizza una CTE ricorsiva per individuare la categoria richiesta e tutte le sue sottocategorie, restituendo gli annunci in vendita che appartengono a una di esse.
- “sp_DettaglioAnnuncio”, “sp_ElencoNoteAnnuncio” e “sp_ElencoCommentiAnnuncio” realizzano complessivamente OP5. Le tre procedure restituiscono rispettivamente le informazioni principali dell’annuncio, le note integrative e i commenti pubblici.
- “sp_ModificaAnnuncio” implementa OP9. Verifica che il richiedente sia l’autore e che l’annuncio sia ancora in vendita, quindi aggiorna titolo, descrizione, prezzo e categoria.
- “sp_ContrassegnaAnnuncioVenduto” implementa OP10. Verifica la proprietà dell’annuncio e ne aggiorna lo stato a Venduto.
- “sp_ElencoAnnunciUtente” implementa OP14 e restituisce gli annunci pubblicati da uno specifico utente.
- “sp_ElencoDestinatariNotificaAnnuncio” è una procedura interna condivisa da OP3, OP9 e OP10. Recupera gli utenti che seguono l’annuncio insieme al recapito scelto da ciascuno. Il risultato viene utilizzato dall’applicazione per simulare le notifiche.

Procedure relative alle note e ai commenti

- “sp_PubblicaNota” implementa OP3. Verifica che il richiedente sia l’autore di un annuncio ancora in vendita, inserisce la nota e, dopo il completamento dell’operazione, restituisce i destinatari della notifica.
- “sp_InserisciCommento” implementa OP6. Inserisce un commento pubblico e restituisce i recapiti degli utenti da notificare. Tra i destinatari sono compresi i follower dell’annuncio e, quando il commento è stato inserito da un altro utente, anche l’autore.

Gli ulteriori controlli sul contenuto testuale e sullo stato dell’annuncio sono applicati dai trigger, in modo che rimangano validi indipendentemente dal punto dal quale viene effettuato l’inserimento.

Procedure relative ai messaggi privati

- “sp_InviaMessaggioPrivato” implementa OP7. Inserisce un messaggio relativo a un annuncio e restituisce il recapito preferito del destinatario. I trigger verificano che i partecipanti siano distinti, che almeno uno di essi sia l’autore dell’annuncio e che RV20 venga rispettata.
- “sp_ElencoConversazioniUtente” restituisce l’elenco delle conversazioni alle quali partecipa l’utente, raggruppandole per annuncio e interlocutore.

- “sp_VisualizzaConversazione” implementa OP15. Prima di restituire i messaggi verifica che il richiedente sia uno dei partecipanti e che uno dei due utenti sia l’autore dell’annuncio. I messaggi vengono ordinati cronologicamente mediante DataOra e, a parità di data, mediante CodiceMessaggio.

Procedure relative al monitoraggio

- “sp_ElencoAnnunciSeguiti” restituisce gli annunci seguiti da uno specifico utente.
- “sp_SeguiAnnuncio” implementa l’iscrizione prevista da OP8. Verifica che l’annuncio sia ancora in vendita e inserisce la relativa riga nella tabella Segui.
- “sp_SmettiDiSeguireAnnuncio” implementa la disiscrizione prevista dalla stessa operazione ed elimina la corrispondente riga da Segui.

I trigger associati alla tabella Segui impediscono inoltre all’autore di seguire il proprio annuncio e rifiutano l’iscrizione a un annuncio già venduto.

Procedure riservate al gestore

- “sp_CreaCategoria” implementa OP11. Inserisce una nuova categoria, eventualmente collegandola a una categoria padre. I trigger verificano il ruolo del creatore, l’assenza di duplicati e la correttezza della gerarchia.
- “sp_GeneraReportUtenti” implementa OP12. Verifica che il richiedente sia un gestore e restituisce le statistiche calcolate dalla vista “v_StatisticheUtenti”.

Gestione delle transazioni

Non tutte le procedure richiedono una transazione esplicita. Le procedure composte da una singola istruzione SQL sfruttano l’atomicità già garantita da MySQL. Sono state invece introdotte transazioni esplicite quando un controllo e la successiva modifica devono costituire un’unica operazione oppure quando più letture logicamente collegate devono essere eseguite nello stesso contesto.

Le procedure transazionali utilizzano un gestore degli errori comune:

```
DECLARE EXIT HANDLER FOR SQLEXCEPTION  
  
BEGIN
```

```
ROLLBACK;  
  
RESIGNAL;  
  
END;
```

Se un'istruzione genera un errore, il gestore esegue ROLLBACK, annullando le modifiche non ancora confermate, e successivamente RESIGNAL, restituendo l'errore al chiamante.

È stato utilizzato il livello di isolamento READ COMMITTED, che impedisce la lettura di modifiche non ancora confermate senza mantenere per tutta la transazione una versione invariabile di ogni dato letto. Questo livello è adeguato alle operazioni dell'applicazione e limita la durata dei blocchi rispetto a livelli più restrittivi.

Transazioni di modifica

“sp_PubblicaNota”, “sp_SeguiAnnuncio”, “sp_ModificaAnnuncio” e “sp_ContrassegnaAnnuncioVenduto” eseguono inizialmente una lettura dell'annuncio mediante SELECT ... FOR UPDATE. La riga viene bloccata fino al COMMIT, impedendo che un'altra transazione ne modifichi contemporaneamente lo stato tra il controllo e l'operazione successiva.

La struttura adottata è la seguente:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
START TRANSACTION;  
  
SELECT ...  
  
    FROM Annuncio  
  
WHERE ...  
  
    FOR UPDATE;  
  
-- Verifica delle condizioni e modifica dei dati
```

```
COMMIT;
```

Questa soluzione evita, ad esempio, che un utente inizi a seguire un annuncio nello stesso momento in cui l'autore lo contrassegna come venduto, oppure che una nota o una modifica venga applicata dopo che lo stato dell'annuncio è cambiato.

In “sp_PubblicaNota”, “sp_ModificaAnnuncio” e “sp_ContrassegnaAnnuncioVenduto”, l'elenco dei destinatari viene recuperato dopo il COMMIT. Le notifiche devono infatti essere elaborate soltanto dopo che la modifica principale è stata confermata. Inoltre, la consultazione dei follower non prolunga inutilmente il blocco sulla riga dell'annuncio.

Transazioni di sola lettura

“sp_GeneraReportUtenti” e “sp_VisualizzaConversazione” utilizzano:

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;  
  
START TRANSACTION READ ONLY;
```

La clausola READ ONLY dichiara che la transazione non deve modificare i dati.

Nel report, la transazione comprende il controllo del profilo del gestore e la successiva lettura delle statistiche. Nella visualizzazione di una conversazione comprende invece la verifica dei partecipanti e il recupero dei messaggi. In entrambi i casi un eventuale errore provoca il ROLLBACK, mentre il completamento corretto termina con COMMIT.

Codice SQL

```
USE BachecaAnnunci;  
  
DELIMITER $$  
  
-- OP1 e RV8: registrazione di un utente ordinario.
```

```
DROP PROCEDURE IF EXISTS sp_RegistraUtente $$

CREATE PROCEDURE sp_RegistraUtente (

    IN p_Username VARCHAR(30),

    IN p_PasswordHash VARCHAR(255),

    IN p_Nome VARCHAR(50),

    IN p_Cognome VARCHAR(50),

    IN p_CodiceFiscale CHAR(16),

    IN p_DataNascita DATE,

    IN p_IndirizzoResidenza VARCHAR(100),

    IN p_IndirizzoFatturazione VARCHAR(100),

    IN p_Email VARCHAR(100),

    IN p_Cellulare VARCHAR(20),

    IN p_TelefonoFisso VARCHAR(20),

    IN p_RecapitoPreferito VARCHAR(15)

)

MODIFIES SQL DATA

-- La procedura accede alle tabelle con i privilegi del creatore

SQL SECURITY DEFINER

BEGIN

    INSERT INTO Utente (

        Username,

        PasswordHash,
```

```
Nome,  
  
Cognome,  
  
CodiceFiscale,  
  
DataNascita,  
  
IndirizzoResidenza,  
  
IndirizzoFatturazione,  
  
Email,  
  
Cellulare,  
  
TelefonoFisso,  
  
RecapitoPreferito,  
  
IsGestore  
  
)  
  
VALUES (  
  
    p_Username,  
  
    p_PasswordHash,  
  
    p_Nome,  
  
    p_Cognome,  
  
    p_CodiceFiscale,  
  
    p_DataNascita,  
  
    p_IndirizzoResidenza,  
  
    p_IndirizzoFatturazione,  
  
    p_Email,
```

```
        p_Cellulare,  
  
        p_TelefonoFisso,  
  
        p_RecapitoPreferito,  
  
        FALSE  
  
    );  
  
END $$  
  
-- Procedura di supporto: elenco delle categorie selezionabili.  
  
DROP PROCEDURE IF EXISTS sp_ElencoCategorie $$  
  
CREATE PROCEDURE sp_ElencoCategorie ()  
  
READS SQL DATA  
  
SQL SECURITY DEFINER  
  
BEGIN  
  
    SELECT  
  
        CodiceCategoria,  
  
        Nome,  
  
        CategoriaPadre,  
  
        GestoreCreatore  
  
    FROM Categoria  
  
    ORDER BY Nome;  
  
END $$
```

```
-- OP2: pubblicazione di un nuovo annuncio.

DROP PROCEDURE IF EXISTS sp_PubblicaAnnuncio $$

CREATE PROCEDURE sp_PubblicaAnnuncio (

    IN p_Titolo VARCHAR(100),

    IN p_DescrizioneArticolo TEXT,

    IN p_Prezzo DECIMAL(10, 2),

    IN p_Autore VARCHAR(30),

    IN p_Categoria INT UNSIGNED

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    INSERT INTO Annuncio (

        Titolo,

        DescrizioneArticolo,

        Prezzo,

        Stato,

        Autore,

        Categoria

    )

    VALUES (

        p_Titolo,
```

```
        p_DescrizioneArticolo,  
  
        p_Prezzo,  
  
        'InVendita',  
  
        p_Autore,  
  
        p_Categoria  
  
    );  
  
END $$  
  
-- Procedura interna condivisa da OP3, OP9 e OP10 (Notifica).  
  
DROP PROCEDURE IF EXISTS sp_ElencoDestinatariNotificaAnnuncio $$  
  
CREATE PROCEDURE sp_ElencoDestinatariNotificaAnnuncio (  
  
    IN p_CodiceAnnuncio INT UNSIGNED  
  
)  
  
READS SQL DATA  
  
SQL SECURITY DEFINER  
  
BEGIN  
  
    SELECT  
  
        r.Username,  
  
        r.RecapitoPreferito,  
  
        r.Recapito  
  
    FROM Segui AS s  
  
    JOIN v_RecapitiPreferiti AS r
```

```
        ON r.Username = s.UsernameUtente

    WHERE s.CodiceAnnuncio = p_CodiceAnnuncio;

END $$

-- OP3: pubblicazione di una nota e recupero dei follower da
-- notificare.

DROP PROCEDURE IF EXISTS sp_PubblicaNota $$

CREATE PROCEDURE sp_PubblicaNota (

    IN p_UsernameAutore VARCHAR(30),

    IN p_CodiceAnnuncio INT UNSIGNED,

    IN p_TestoNota TEXT

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE var_CodiceAnnuncio INT UNSIGNED DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;
```

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

START TRANSACTION;

SELECT Codice
INTO var_CodiceAnnuncio
FROM Annuncio
WHERE Codice = p_CodiceAnnuncio
      AND Autore = p_UsernameAutore
      AND Stato = 'InVendita'
FOR UPDATE;

IF var_CodiceAnnuncio IS NULL THEN
    SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'RV11: solo 1 autore puo integrare un
proprio annuncio in vendita';
END IF;

INSERT INTO Nuova_Nota (
    TestoNota,
    DataOra,
    Annuncio
```

```
)

VALUES (

    p_TestoNota,

    CURRENT_TIMESTAMP,

    var_CodiceAnnuncio

);

COMMIT;

CALL sp_ElencoDestinatariNotificaAnnuncio(p_CodiceAnnuncio);

END $$

-- OP4: recupero degli annunci di una categoria e delle sue
sottocategorie.

DROP PROCEDURE IF EXISTS sp_RicercaAnnunciPerCategoria $$

CREATE PROCEDURE sp_RicercaAnnunciPerCategoria (

    IN p_CodiceCategoria INT UNSIGNED,

    IN p_UsernameRichiedente VARCHAR(30)

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN
```

```
WITH RECURSIVE SottoCategorie AS (  
  
    SELECT CodiceCategoria  
  
    FROM Categoria  
  
    WHERE CodiceCategoria = p_CodiceCategoria  
  
    UNION ALL  
  
    SELECT c.CodiceCategoria  
  
    FROM Categoria AS c  
  
    JOIN SottoCategorie AS sc  
        ON c.CategoriaPadre = sc.CodiceCategoria  
)  
  
SELECT  
  
    a.Codice,  
  
    a.Titolo,  
  
    a.Prezzo,  
  
    a.Stato,  
  
    a.Autore,  
  
    a.Categoria,  
  
    c.Nome AS NomeCategoria  
  
FROM Annuncio AS a  
  
JOIN SottoCategorie AS sc
```

```
        ON sc.CodiceCategoria = a.Categoria

JOIN Categoria AS c

        ON c.CodiceCategoria = a.Categoria

WHERE a.Stato = 'InVendita'

        AND a.Autore <> p_UsernameRichiedente

ORDER BY a.Titolo;

END $$

-- OP5: visualizzazione della scheda completa di un annuncio.

DROP PROCEDURE IF EXISTS sp_DettaglioAnnuncio $$

CREATE PROCEDURE sp_DettaglioAnnuncio (

    IN p_CodiceAnnuncio INT UNSIGNED

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    SELECT *

    FROM v_DettaglioAnnuncio

    WHERE Codice = p_CodiceAnnuncio;

END $$

-- OP5: elenco delle note integrative di un annuncio.
```

```
DROP PROCEDURE IF EXISTS sp_ElencoNoteAnnuncio $$

CREATE PROCEDURE sp_ElencoNoteAnnuncio (

    IN p_CodiceAnnuncio INT UNSIGNED

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    SELECT

        CodiceNota,

        TestoNota,

        DataOra

    FROM Nuova_Nota

    WHERE Annuncio = p_CodiceAnnuncio

    ORDER BY DataOra;

END $$

-- OP5: elenco dei commenti pubblici di un annuncio.

DROP PROCEDURE IF EXISTS sp_ElencoCommentiAnnuncio $$

CREATE PROCEDURE sp_ElencoCommentiAnnuncio (

    IN p_CodiceAnnuncio INT UNSIGNED

)

READS SQL DATA
```

```
SQL SECURITY DEFINER

BEGIN

    SELECT

        CodiceCommento,

        Testo,

        DataOra,

        Autore

    FROM Commento_Pubblico

    WHERE Annuncio = p_CodiceAnnuncio

    ORDER BY DataOra;

END $$

-- OP6: inserimento di un commento pubblico.

DROP PROCEDURE IF EXISTS sp_InserisciCommento $$

CREATE PROCEDURE sp_InserisciCommento (

    IN p_Testo TEXT,

    IN p_CodiceAnnuncio INT UNSIGNED,

    IN p_Autore VARCHAR(30)

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN
```

```
INSERT INTO Commento_Pubblico (  
  
    Testo,  
  
    DataOra,  
  
    Annuncio,  
  
    Autore  
  
)  
  
VALUES (  
  
    p_Testo,  
  
    CURRENT_TIMESTAMP,  
  
    p_CodiceAnnuncio,  
  
    p_Autore  
  
);  
  
  
SELECT  
  
    r.Username,  
  
    r.RecapitoPreferito,  
  
    r.Recapito  
  
FROM v_RecapitiPreferiti AS r  
  
JOIN (  
  
    SELECT s.UsernameUtente AS Username  
  
    FROM Segui AS s  
  
    WHERE s.CodiceAnnuncio = p_CodiceAnnuncio
```

```
        AND s.UsernameUtente <> p_Autore

    UNION ALL

    SELECT a.Autore AS Username

    FROM Annuncio AS a

    WHERE a.Codice = p_CodiceAnnuncio

        AND a.Autore <> p_Autore

    ) AS destinatari

    ON destinatari.Username = r.Username;

END $$

-- OP7: invio di un messaggio privato relativo a un annuncio.

DROP PROCEDURE IF EXISTS sp_InviaMessaggioPrivato $$

CREATE PROCEDURE sp_InviaMessaggioPrivato (

    IN p_Testo TEXT,

    IN p_CodiceAnnuncio INT UNSIGNED,

    IN p_Mittente VARCHAR(30),

    IN p_Destinataro VARCHAR(30)

)

MODIFIES SQL DATA

SQL SECURITY DEFINER
```

```
BEGIN

    INSERT INTO Messaggio_Privato (

        Testo,

        DataOra,

        Annuncio,

        Mittente,

        Destinatario

    )

    VALUES (

        p_Testo,

        CURRENT_TIMESTAMP,

        p_CodiceAnnuncio,

        p_Mittente,

        p_Destinatarior

    );

SELECT

    Username,

    RecapitoPreferito,

    Recapito

FROM v_RecapitiPreferiti

WHERE Username = p_Destinatarior;
```

```
END $$

-- Procedura di supporto a OP8: annunci gia seguiti dall'utente.

DROP PROCEDURE IF EXISTS sp_ElencoAnnunciSeguiti $$

CREATE PROCEDURE sp_ElencoAnnunciSeguiti (

    IN p_username VARCHAR(30)

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    SELECT

        d.Codice,

        d.Titolo,

        d.DescrizioneArticolo,

        d.Prezzo,

        d.Stato,

        d.Autore,

        d.CodiceCategoria,

        d.NomeCategoria

    FROM Segui AS s

    JOIN v_DettaglioAnnuncio AS d

        ON d.Codice = s.CodiceAnnuncio
```

```
WHERE s.UsernameUtente = p_Username

ORDER BY d.Titolo;

END $$

-- OP8: iscrizione al monitoraggio di un annuncio.

DROP PROCEDURE IF EXISTS sp_SeguiAnnuncio $$

CREATE PROCEDURE sp_SeguiAnnuncio (

    IN p_Username VARCHAR(30),

    IN p_CodiceAnnuncio INT UNSIGNED

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE var_CodiceAnnuncio INT UNSIGNED DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
START TRANSACTION;

SELECT Codice

INTO var_CodiceAnnuncio

FROM Annuncio

WHERE Codice = p_CodiceAnnuncio

      AND Stato = 'InVendita'

FOR UPDATE;

IF var_CodiceAnnuncio IS NULL THEN

      SIGNAL SQLSTATE '45000'

          SET MESSAGE_TEXT = 'RV13: si possono seguire solo
annunci in vendita';

END IF;

INSERT INTO Segui (

      UsernameUtente,

      CodiceAnnuncio

)

VALUES (

      p_Username,

      var_CodiceAnnuncio
```

```
) ;

COMMIT;

END $$

-- OP8: disiscrizione dal monitoraggio di un annuncio.

DROP PROCEDURE IF EXISTS sp_SmettiDiSeguireAnnuncio $$

CREATE PROCEDURE sp_SmettiDiSeguireAnnuncio (

    IN p_Username VARCHAR(30),

    IN p_CodiceAnnuncio INT UNSIGNED

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    DELETE FROM Segui

    WHERE UsernameUtente = p_Username

        AND CodiceAnnuncio = p_CodiceAnnuncio;

END $$

-- OP9: modifica delle informazioni di un proprio annuncio attivo.

DROP PROCEDURE IF EXISTS sp_ModificaAnnuncio $$

CREATE PROCEDURE sp_ModificaAnnuncio (
```

```
IN p_UsernameAutore VARCHAR(30),

IN p_CodiceAnnuncio INT UNSIGNED,

IN p_Titolo VARCHAR(100),

IN p_DescrizioneArticolo TEXT,

IN p_Prezzo DECIMAL(10, 2),

IN p_Categoria INT UNSIGNED

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE var_CodiceAnnuncio INT UNSIGNED DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    START TRANSACTION;

    SELECT Codice
```

```
INTO var_CodiceAnnuncio

FROM Annuncio

WHERE Codice = p_CodiceAnnuncio

      AND Autore = p_UsernameAutore

      AND Stato = 'InVendita'

FOR UPDATE;

IF var_CodiceAnnuncio IS NULL THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'RV11: solo 1 autore puo modificare
un proprio annuncio in vendita';

END IF;

UPDATE Annuncio

SET Titolo = p_Titolo,

      DescrizioneArticolo = p_DescrizioneArticolo,

      Prezzo = p_Prezzo,

      Categoria = p_Categoria

WHERE Codice = var_CodiceAnnuncio;

COMMIT;
```

```
CALL sp_ElencoDestinatariNotificaAnnuncio(p_CodiceAnnuncio);

END $$

-- OP10: aggiornamento di un proprio annuncio allo stato Venduto.

DROP PROCEDURE IF EXISTS sp_ContrassegnaAnnuncioVenduto $$

CREATE PROCEDURE sp_ContrassegnaAnnuncioVenduto (

    IN p_UsernameAutore VARCHAR(30),

    IN p_CodiceAnnuncio INT UNSIGNED

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE var_CodiceAnnuncio INT UNSIGNED DEFAULT NULL;

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    START TRANSACTION;
```

```
SELECT Codice

INTO var_CodiceAnnuncio

FROM Annuncio

WHERE Codice = p_CodiceAnnuncio

      AND Autore = p_UsernameAutore

      AND Stato = 'InVendita'

FOR UPDATE;

IF var_CodiceAnnuncio IS NULL THEN

      SIGNAL SQLSTATE '45000'

      SET MESSAGE_TEXT = 'RV11: solo 1 autore puo vendere un
proprio annuncio in vendita';

END IF;

UPDATE Annuncio

SET Stato = 'Venduto'

WHERE Codice = var_CodiceAnnuncio;

COMMIT;

CALL sp_ElencoDestinatariNotificaAnnuncio(p_CodiceAnnuncio);
```

```
END $$

-- OP11: creazione e collocazione di una categoria.

DROP PROCEDURE IF EXISTS sp_CreaCategoria $$

CREATE PROCEDURE sp_CreaCategoria (

    IN p_Nome VARCHAR(50),

    IN p_CategoriaPadre INT UNSIGNED,

    IN p_GestoreCreatore VARCHAR(30)

)

MODIFIES SQL DATA

SQL SECURITY DEFINER

BEGIN

    INSERT INTO Categoria (

        Nome,

        CategoriaPadre,

        GestoreCreatore

    )

    VALUES (

        p_Nome,

        p_CategoriaPadre,

        p_GestoreCreatore

    );
```

```
END $$

-- OP12: generazione del report statistico riservato ai gestori.

DROP PROCEDURE IF EXISTS sp_GeneraReportUtenti $$

CREATE PROCEDURE sp_GeneraReportUtenti (

    IN p_UsernameGestore VARCHAR(30)

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    START TRANSACTION READ ONLY;

    IF NOT EXISTS (

        SELECT 1

        FROM Utente
```

```
WHERE Username = p_UsernameGestore

AND IsGestore = TRUE

) THEN

SIGNAL SQLSTATE '45000'

SET MESSAGE_TEXT = 'RV12: il report e riservato ai
gestori';

END IF;

SELECT

Username,

AnnunciTotali,

AnnunciVenduti,

PercentualeVenduti

FROM v_StatisticheUtenti

ORDER BY Username;

COMMIT;

END $$

-- OP13: recupero dei dati necessari all'autenticazione e alla
sessione.

DROP PROCEDURE IF EXISTS sp_RecuperaProfiloAccesso $$
```

```
CREATE PROCEDURE sp_RicuperaProfiloAccesso (  
    IN p_Username VARCHAR(30)  
)  
  
READS SQL DATA  
  
SQL SECURITY DEFINER  
  
BEGIN  
  
    SELECT  
  
        Username,  
  
        PasswordHash,  
  
        Nome,  
  
        Cognome,  
  
        IsGestore  
  
    FROM Utente  
  
    WHERE Username = p_Username;  
  
END $$  
  
-- OP14: visualizzazione degli annunci pubblicati da un utente.  
  
DROP PROCEDURE IF EXISTS sp_ElencoAnnunciUtente $$  
  
CREATE PROCEDURE sp_ElencoAnnunciUtente (  
    IN p_Username VARCHAR(30)  
)  
  
READS SQL DATA
```

```
SQL SECURITY DEFINER

BEGIN

    SELECT

        Codice,

        Titolo,

        DescrizioneArticolo,

        Prezzo,

        Stato,

        CodiceCategoria,

        NomeCategoria

    FROM v_DettaglioAnnuncio

    WHERE Autore = p_Username

    ORDER BY Titolo;

END $$

-- OP15: visualizzazione della conversazione privata relativa a un
annuncio.

DROP PROCEDURE IF EXISTS sp_ElencoConversazioniUtente $$

CREATE PROCEDURE sp_ElencoConversazioniUtente (

    IN p_Username VARCHAR(30)

)

READS SQL DATA
```

```
SQL SECURITY DEFINER

BEGIN

    SELECT DISTINCT

        m.Annuncio AS CodiceAnnuncio,

        a.Titolo AS TitoloAnnuncio,

        IF(

            m.Mittente = p_Username,

            m.Destinatario,

            m.Mittente

        ) AS AltroUtente

    FROM Messaggio_Privato AS m

    JOIN Annuncio AS a

        ON a.Codice = m.Annuncio

    WHERE m.Mittente = p_Username

        OR m.Destinatario = p_Username

    ORDER BY TitoloAnnuncio, AltroUtente;

END $$

-- OP15: visualizzazione della conversazione privata relativa a un
annuncio.

DROP PROCEDURE IF EXISTS sp_VisualizzaConversazione $$

CREATE PROCEDURE sp_VisualizzaConversazione (
```

```
    IN p_UsernameRichiedente VARCHAR(30),

    IN p_AltroUtente VARCHAR(30),

    IN p_CodiceAnnuncio INT UNSIGNED
)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE EXIT HANDLER FOR SQLEXCEPTION

    BEGIN

        ROLLBACK;

        RESIGNAL;

    END;

    SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

    START TRANSACTION READ ONLY;

    IF p_UsernameRichiedente = p_AltroUtente THEN

        SIGNAL SQLSTATE '45000'

            SET MESSAGE_TEXT = 'RV4: mittente e destinatario devono
essere distinti';

    END IF;
```

```
IF NOT EXISTS (  
  
    SELECT 1  
  
    FROM Annuncio  
  
    WHERE Codice = p_CodiceAnnuncio  
  
        AND Autore IN (p_UsernameRichiedente, p_AltroUtente)  
  
) THEN  
  
    SIGNAL SQLSTATE '45000'  
  
        SET MESSAGE_TEXT = 'RV6: uno dei partecipanti deve  
essere autore dell annuncio';  
  
END IF;  
  
SELECT  
  
    CodiceMessaggio,  
  
    Testo,  
  
    DataOra,  
  
    Mittente,  
  
    Destinatario  
  
FROM Messaggio_Privato  
  
WHERE Annuncio = p_CodiceAnnuncio  
  
    AND (  
  
        (Mittente = p_UsernameRichiedente AND Destinatario =  
p_AltroUtente)
```

```
OR

        (Mittente = p_AltroUtente AND Destinatario =
p_UsernameRichiedente)

    )

    ORDER BY DataOra, CodiceMessaggio;

COMMIT;

END $$

-- Controllo interno: coerenza tra account DBMS e profilo
applicativo.

DROP PROCEDURE IF EXISTS sp_VerificaProfiloSessione $$

CREATE PROCEDURE sp_VerificaProfiloSessione (

    IN p_Username VARCHAR(30)

)

READS SQL DATA

SQL SECURITY DEFINER

BEGIN

    DECLARE var_IsGestore BOOLEAN DEFAULT NULL;

    DECLARE var_AccountDBMS VARCHAR(100);

    SELECT IsGestore
```

```
INTO var_IsGestore

FROM Utente

WHERE Username = p_Username;

IF var_IsGestore IS NULL THEN

    SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Sessione non valida: utente
inesistente';

END IF;

SET var_AccountDBMS = SUBSTRING_INDEX(USER(), '@', 1);

IF NOT (

    (var_AccountDBMS = 'account_utente' AND var_IsGestore =
FALSE)

    OR

    (var_AccountDBMS = 'account_gestore' AND var_IsGestore =
TRUE)

) THEN

    SIGNAL SQLSTATE '45000'

        SET MESSAGE_TEXT = 'Sessione non valida: profilo e ruolo
DBMS non coerenti';

END IF;
```

```
SELECT

    p_username AS Username,

    IF(var_IsGestore, 'Gestore', 'Utente') AS ProfiloVerificato;

END $$

DELIMITER ;
```